Cloud computing

Course notes

Daniele Besozzi

A.A. 26/27

0 Contents

1 Introduction and definition 4
1.1 Essential characteristics 4
1.2 Why cloud computing? 5
1.3 Data centers and cloud computing 5
1.4 Virtual infrastructure manager (VIM) 6
2 Service models (SPI) 7
2.1 Infrastructure as a Service (IaaS) 7
2.1.1 Resource Management Interface 7
2.1.2 System Monitoring Interface 8
2.1.3 Properties of IaaS 8
2.2 Platform as a Service (PaaS) 8
2.2.1 Properties of PaaS 9
2.3 Software as a Service (SaaS) 11
2.3.1 Properties of SaaS 12
2.4 The SPI model 12
3 Deployment models 14
3.1 Public Cloud 14
3.2 Private Cloud 14
3.3 Community Cloud 14
3.4 Hybrid Cloud 14
4 Characteristics 15
4.1 Utility Computing 15
4.1.1 Service oriented architecture (SOA) 16
4.2 Scalability and Elasticity 16
4.2.1 Traditional provisioning 17
4.2.2 Dynamic provisioning 17
4.3 Availability and Reliability 18
4.3.1 Fault tolerance 18
4.3.2 System resilience 19
4.3.3 System Security 19
4.4 Manageability & Interoperability 19
4.4.1 System Monitoring 19
4.4.2 Billing System 20
4.5 Performance optimization 20
4.5.1 Economical aspects of cloud computing 20
5 Virtualization 21
5.1 Computer architecture 21
5.1.1 Emulation 21
5.2 Virtualization and Virtual Machines 21
5.2.1 Protection rings 22
5.3 Virtualization of resources 23
5.3.1 Memory virtualization 23
5.3.2 I/O virtualization 24
5.3.3 Network virtualization 24
5.4 Advantages of virtualization 24

5.4.1 Workload isolation 24
5.4.2 Workload migration 25
5.4.3 Consolidation 25
5.5 Taxonomy of hypervisors 25
5.6 Virtualization techniques 25
5.6.1 Full Virtualization using binary translation 25
5.6.2 Para Virtualization (a.k.a. OS assisted Virtualization) 26
5.6.3 OS-Level virtualization (Containerization) 26
6 deep dive: IaaS (AWS) 27
6.1 Key concepts 27
6.1.1 S3 27
6.1.2 EBS 27
6.1.3 Security group 28
6.2 EC2 28
6.2.1 Ephemeral resources 28
6.2.2 Persistent Resources 29
6.2.3 Pricing model 29
6.2.3.1 Purchase options 29
6.3 Autoscaling 30
6.3.1 Layered architecture 31
6.3.2 Load balancing 31
6.3.3 Amazon Auto Scaling group 31
6.3.3.1 Launch Configuration 31
6.3.4 Auto Scaling with AWS 31
6.3.4.1 Health checks 33
6.3.4.2 Triggers in Auto Scaling Systems 33
6.3.4.3 Fleet management 34
6.3.4.4 Cooldown — Stabilizing the Scaling Feedback Loop 34
7 deep dive: PaaS (Azure) 35
7.1 Azure Web App service 36
7.1.1 Roles 36
7.1.1.1 Web role 36
7.1.1.2 Worker Role 37
7.2 Azure Auto scaling Application Block 37
7.2.1 Throttling 38
7.3 Storage for persistent data 38
7.3.1 Azure SQL Database 38
7.3.2 CosmoDB 39
8 Serverless computing 40
8.1 Backend as a Service (BaaS) 40
8.2 Function as a Service (FaaS) 41
8.2.1 FaaS enabler: API Gateway 42
8.2.2 State 42
8.2.3 Time (duration and startup) 42
8.2.4 pros and cons 43
8.3 Amazon Lambda 44
8.4 Serverless vs PaaS 45
8.4.1 Level of Abstraction 45

8.4.2 Scaling 45
8.4.3 Billing 45
8.5 summary 45
9 Towards a world of services 46
9.1 Internet of Things (IoT) 46
9.2 Smart systems 46
9.2.1 Service computing 47
9.2.2 Software ecosystem 47
9.2.3 Semantic Annotations 48
9.3 Edge vs. Cloud computing 48
9.4 API economy 48
9.5 System of systems 50
9.5.1 The Systems of Record 51
9.5.2 Systems of Engagement 51
9.5.3 Systems of insight 52
9.6 Service science 53

1 Introduction and definition

We can define cloud computing in many ways, but the most important thing is to understand the basic definition. A Cloud is a type of distributed system consisting of:

- **interconnected** and **virtualized** computers
- **provisioned on demand** and
- **accessible** through a network.

Alternatively, we can say that cloud computing is:

- a **user experience**, in a sense that it is a way to provide services to users over the network, and
- an **infrastructure management methodology**, because it is a way to manage large numbers of highly virtualized resources so that they can be automatically aggregated to deliver services.

We can give a more formal definition by the NIST (National Institute of Standards and Technology):

> **Definition 1.1** *(Cloud computing).* Cloud computing is a **model** for enabling **convenient**, **on-demand network access** to a **shared** pool of **configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released with minimal management** effort or service provider interaction.

This cloud model promotes availability and is composed of:

- five essential characteristics,
- three service models, and
- four deployment models.

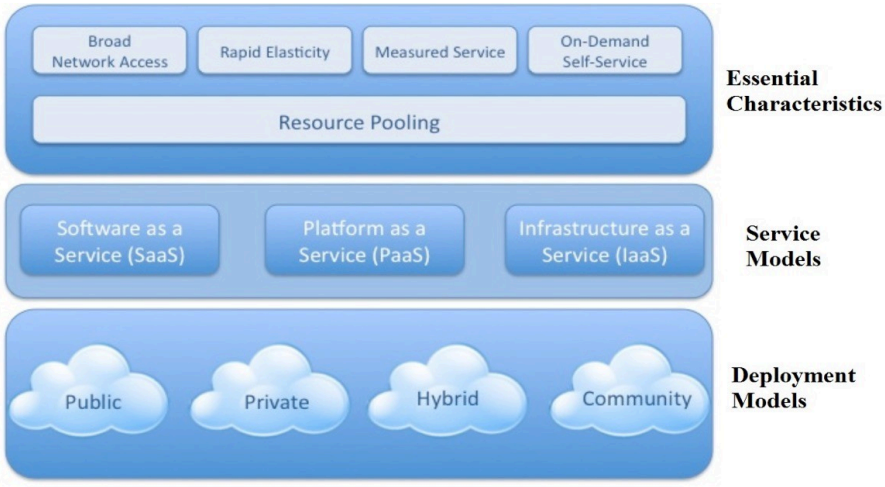


Figure 1: Cloud computing characteristics and models

This gives us a technological, economical, and societal model.

1.1 Essential characteristics

The essential characteristics of cloud computing are:

- **On-demand self-service**:
  - A consumer can unilaterally provision computing capabilities such as server time and network storage as needed automatically, without (or with little) requiring human interaction with a cloud provider.

- **Broad network access**:
  - Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and PDAs) as well as other traditional or cloud- based software services.
- **Resource pooling**:
  - The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.
- **Rapid elasticity**:
  - Capabilities can be rapidly and elastically provisioned to quickly scale up; and rapidly released to quickly scale down, automatically.
  - To consumers, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
- **Measurable service**:
  - Cloud systems automatically control and optimize resource usage by leveraging a metering capability at some level of abstraction appropriate to the type of service.
  - Resource usage can be monitored, controlled, and reported - providing transparency for both the provider and consumer of the service.

1.2 Why cloud computing?

The cloud shape is often used in network diagrams to represent a "black box" network. The access **interfaces are known, but not the internal structure**. A cloud is opaque and considered very large and distant. As in the sky there are **different types of clouds** so there are **different types of services** and **features** offered in the cloud. The internal implementation of these services is **opaque**.

Clouds may overlap, that is, different models of clouds can be implemented by the same vendor. Clouds can **dynamically change shape**. Applications and services in cloud can dynamically **adapt**.

1.3 Data centers and cloud computing

Data centers are buildings that house hundreds of thousands of servers that simultaneously support many services and applications. Data centers are built on a large scale to reduce maintenance costs, but not very large (also to limit blast radius).

Data centers exploit **virtualization** massively to run applications.

A data center requires:

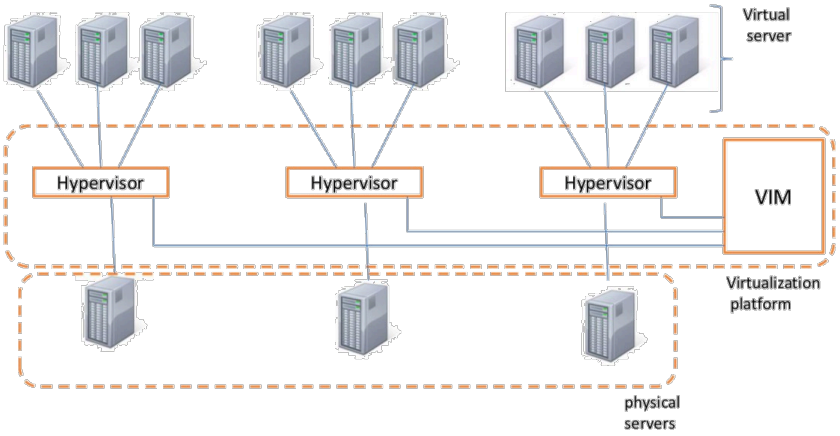
- **Power supply** (generator system to overcome possible power failures).
- Continuous **cooling** system.
- **Security system** (fire alarms, adequate ventilation to eliminate smoke, authorized access, video cameras, security code access, biometric access, card reader access, locked cabinets…).
- **High-speed network**, within the data center, between data centers and to the Internet Often optical fiber, dedicated and proprietary lines, including intercontinental lines.

> **Definition 1.2** *(Availability zone).* Every Availability Zone is an independent data center with its own power grid and network so that it is protected from possible failures in other data centers

Definition 1.3 *(Region).* A Region is a set of Availability Zones that are geographically close to each other and connected by low-latency links. A Region is isolated from other Regions, so that a failure in one Region does not affect other Regions

1.4 Virtual infrastructure manager (VIM)

A key challenge cloud providers face in building an infrastructure is managing physical and virtual resources, i.e., servers, storage and networks, holistically. **Resource orchestration** must be performed in a way that quickly and dynamically delivers resources to users and applications. The software tool responsible for the orchestration is called a **Virtual Infrastructure Manager (VIM)**. This type of software resembles a traditional operating system ("cloud operating system") - but instead of dealing with a single computer, it aggregates resources from multiple computers, presenting a uniform view to the user and applications.



2 Service models (SPI)

A service model is, in this scenario, a high-level characterization (model) of the cloud that **focuses primarily on the services offered and the type of users**. The set of service models defines a possible **taxonomy**.

2.1 Infrastructure as a Service (IaaS)

The capability provided to the consumer (user or application) is to provision:

- virtual machines,
- storage,
- networks, and
- other fundamental computing resources

The consumer can deploy and run arbitrary software on the virtualized resources, which can include OS and applications. The consumer **does not manage or control the underlying cloud infrastructure** but has **control over operating systems, storage, deployed applications**, and possibly limited control of selected networking components.

This kind of technique is enabled by **virtualization**.

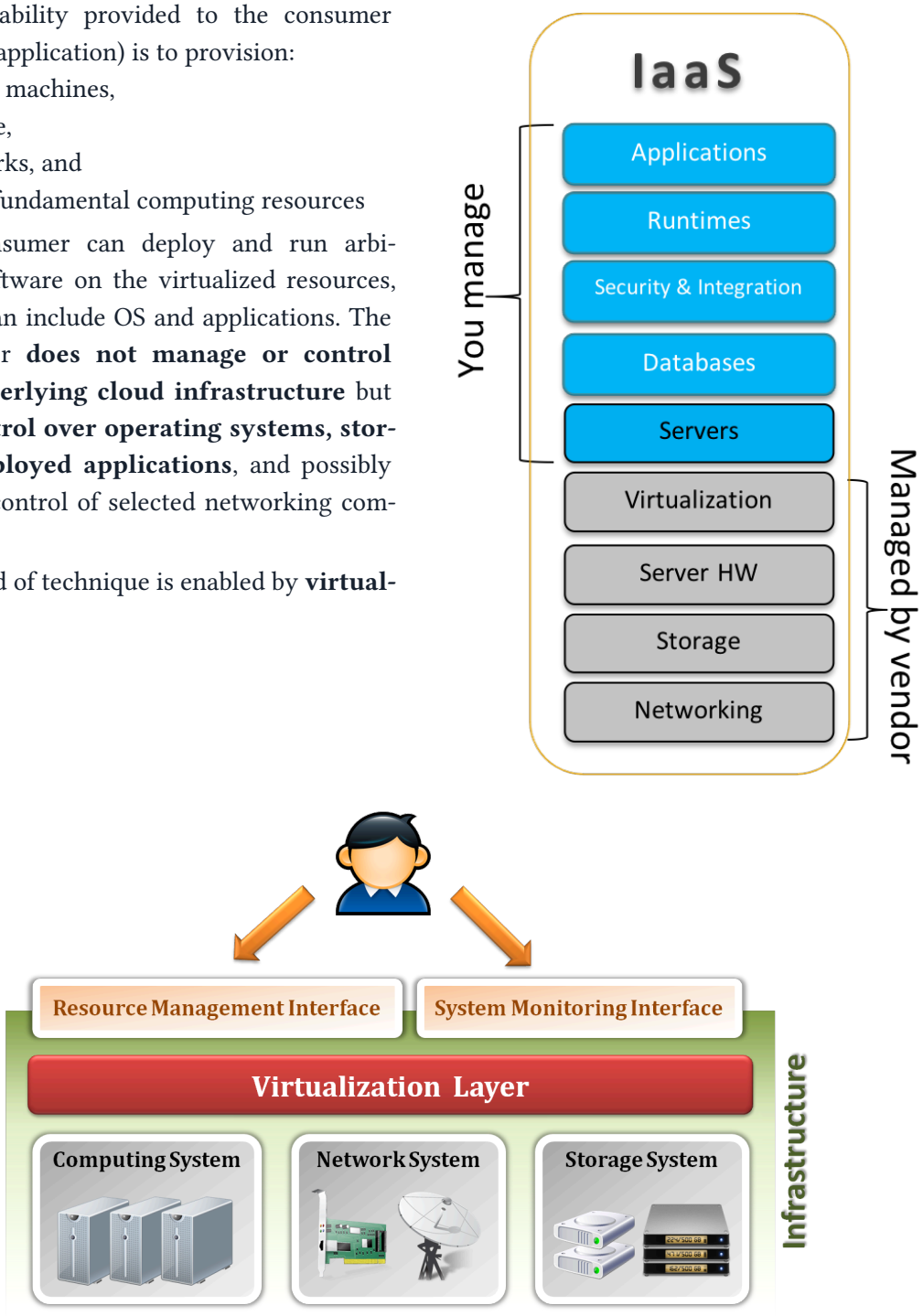


Figure 4: IAAS architecture

2.1.1 Resource Management Interface

RMI Enables the customer to obtain and manage several types of virtualized resource:

- **Virtual Machine**: IaaS provides the basic virtual machine operations, such as creation, suspension, resumption and termination.

- **Virtual Storage**: IaaS provides the basic virtual storage operations, such as space allocation, space release, data writing and data reading.
- **Virtual Network**: IaaS provides the basic virtual network operations, such as IP address allocation, domain name register, connection establishment and bandwidth provision.

2.1.2 System Monitoring Interface

There are several metrics to be monitored:

- **Virtual Machine**: As an IaaS provider, we should be able to monitor some system states of each virtual machine, such as CPU loading,, memory utilization, IO loading and internal network loading
- **Virtual Storage**: As an IaaS provider, we should be able to monitor some storage states of each virtual storage, such as virtual space utilization, data duplication and storage device access bandwidth, ...etc.
- **Virtual Network**: As an IaaS provider, we should be able to monitor some network states of each virtual network, such as virtual network bandwidth, network connectivity and network load balancing

2.1.3 Properties of IaaS

- Manageability and Interoperability
  - System control, billing, API
- Availability and Reliability
  - Multiple copies of VMs, storage redundancy
- Scalability and Elasticity
  - On demand resource (ex. VMs) provisioning

> This technique is enabled by **virtualization**



Figure 5: Properties of IaaS

2.2 Platform as a Service (PaaS)

The capability provided to the consumer is to **develop and deploy** onto the cloud applications created using **programming languages and tools supported by the provider**. The **consumer** does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has **control over the deployed applications and possibly**

application hosting environment configurations.

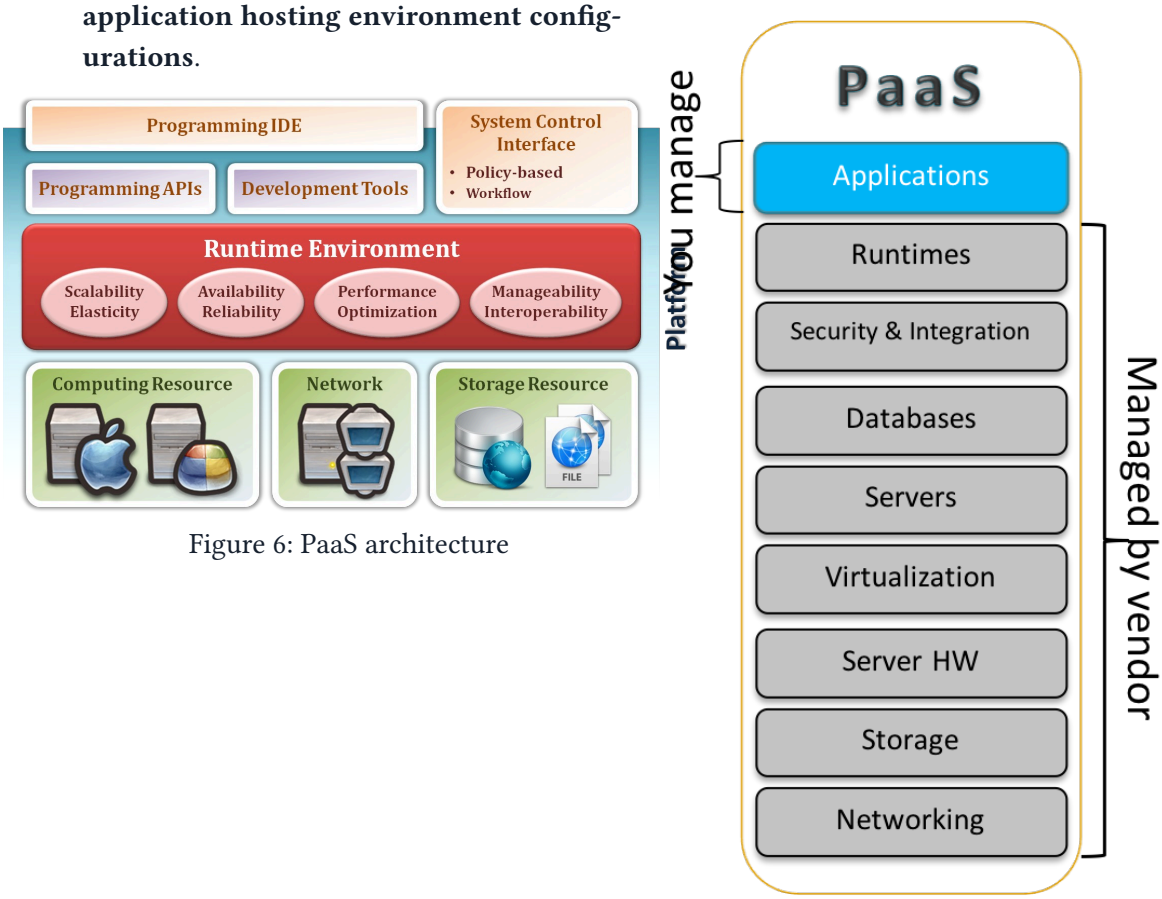


Figure 6: PaaS architecture

> This technique is enabled by **runtime environment**, that is a collection of software services available. Usually implemented by a collection of program libraries and an application execution engine

PAAS offers different services:

- Users make use of **programming IDE** to **develop their service** over PaaS.
  - This IDE should integrate the full functionalities supported from the underling runtime environment. This IDE should also provide development tools, such as profiler, debugger and testing environment.
  - The programming APIs supported from runtime environment may be heterogeneous across different cloud providers, but there are still some common operating functions.
- **System Control Interface**
  - **Policy-Based Control**
    - Typically described as a principle or rule to guide decisions and achieve rational outcome(s)
    - Make the decision according to some requirements
    - If <condition> then <action>
  - **Workflow Control**
    - Describes the flow of installation and configuration of resources
    - Delivers speedy and efficient construction and management of cloud resources

2.2.1 Properties of PaaS

Common properties in Runtime Environment :

- Manageability and Interoperability
- Performance and Optimization
- Availability and Reliability

- Scalability and Elasticity



Figure 8: Properties of PaaS

2.3 Software as a Service (SaaS)

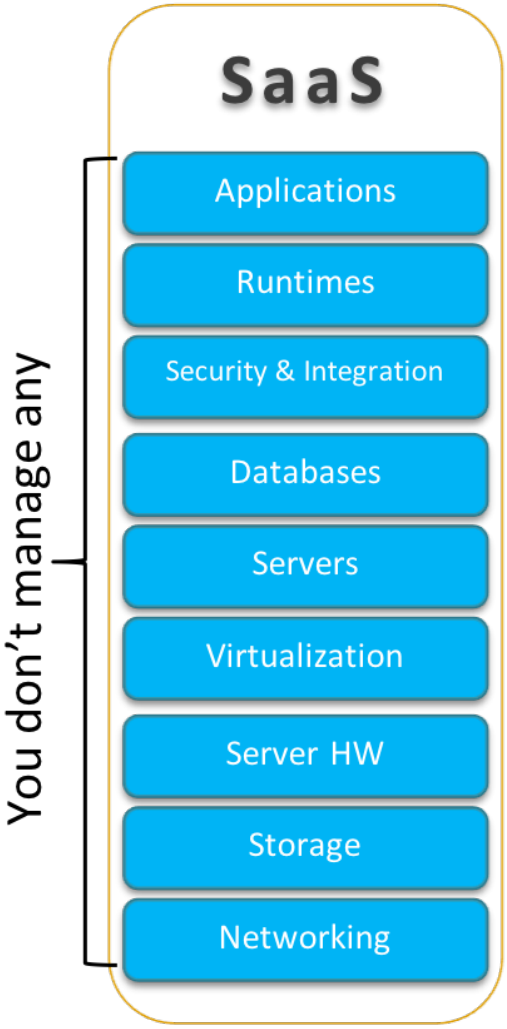
The capability provided to the consumer is to use the **provider's applications running on a cloud infrastructure**. The applications are accessible from various client devices through a thin client interface such as a web browser (e.g., web- based email).

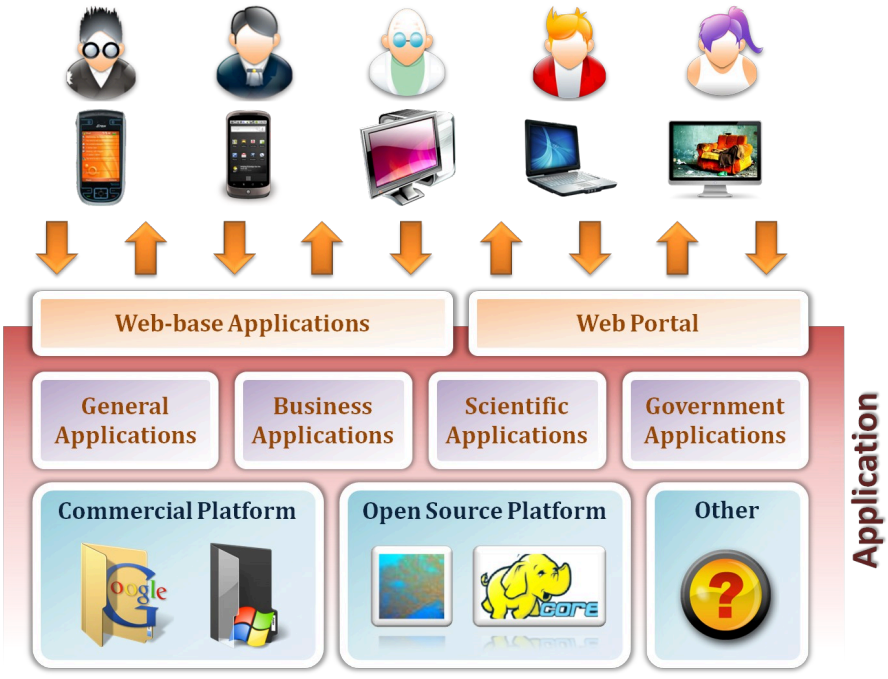
The **consumer does not manage or control the underlying cloud infrastructure** including network, servers, operating systems, storage, or individual application capabilities, except for limited user-specific application configuration settings.

> This technique is enabled by **web services**

Web 2.0 is the trend of using the full potential of the web:

- Viewing the Internet as a computing platform
- Running interactive applications through a web browser
- Leveraging interconnectivity and mobility of devices
- Enhanced effectiveness with greater human participation





Services can be provided in many forms:

- **web-base applications**. Conventional applications should translate their access interface onto web-based platform. Applications can have different domains:
  - General Applications : Applications which are designed for general propose, such as office suit, multimedia and

  instant message.
  - Business Applications : Application which are designed for business propose, such as ERP, CRM and market

  trading system.
  - Scientific Applications : Application which are designed for scientific propose, such as aerospace simulation and

  biochemistry simulation.
  - Government Applications : Applications which are designed for government propose, such as national medical

  system and public transportation system service.
- **Web portal**: Apart from the standard search engine feature, web portals offer other services such as e-mail, news, stock prices, information, databases and entertainment. Portals provide a way for enterprises to provide a consistent look and feel with access control and procedures for multiple applications and databases, which otherwise would have been different entities altogether.

2.3.1 Properties of SaaS

The properties offered by SaaS are the ones offered by the internet, that is **accessibility** and **portability**.

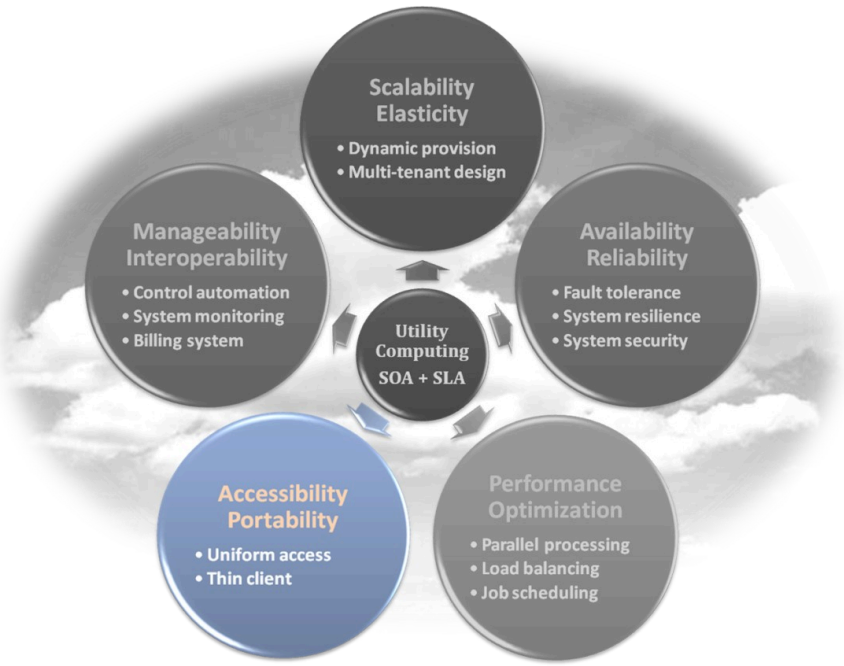


Figure 11: Properties of SaaS

2.4 The SPI model

The SPI model (SaaS, PaaS, IaaS) is the most common classification of the cloud services. But, be aware that the **real cloud is blurry**. The provider can offer different services at different levels, and it is not always easy to classify them.

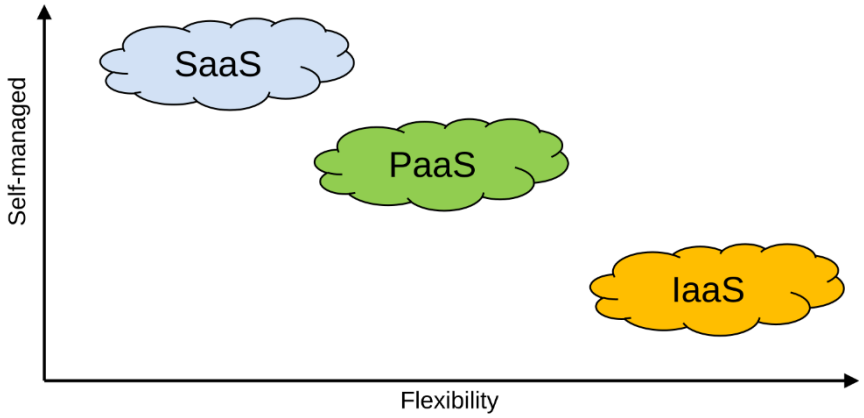


Figure 12: The SPI model

3 Deployment models

A cloud deployment model is defined according to **where the infrastructure resides** and **who has control over that infrastructure**. Deciding which deployment model you will go with is one of the most important cloud deployment decisions you will make.

There are four primary cloud models:

- Public Cloud
- Private Cloud
- Community Cloud
- Hybrid Cloud

Differences lie primarily in the **access**, **size** and **ownership** of published cloud services

3.1 Public Cloud

The cloud infrastructure is made **available to the general public** and is owned by an **organization selling cloud services**. Also known as external cloud or multi-tenant cloud, this model essentially represents a cloud environment that is openly accessible. Basic characteristics:

- **Homogeneous** infrastructure
- Common policies
- Shared resources and multi-tenancy
- Leased or rented infrastructure
- Economies of scale

3.2 Private Cloud

The cloud infrastructure is **operated solely for one organization**. It may be managed by the same organization or a third party and may exist **on premise or off premise**.

Also referred to as internal cloud or on-premise cloud, a private cloud intentionally limits access to its resources to consumers that belong to the organization that owns the cloud. Basic characteristics :

- **Heterogeneous** infrastructure
- Customized and tailored policies
- Dedicated resources
- In-house infrastructure
- End-to-end control

3.3 Community Cloud

The cloud infrastructure is **shared by several organizations** It supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). Access to a community cloud environment is **typically restricted to the members of the community**.

3.4 Hybrid Cloud

The cloud infrastructure is a **composition** of two or more clouds (private, community, or public) that remain unique entities but are **bound together by standardized or proprietary technology** that enables data and application portability (e.g., cloud bursting for load-balancing between clouds). Typical example: Private-Public for scalability.

4 Characteristics

Cloud computing is a paradigm of computing, a new way of thinking about IT industry but **not any specific technology**.

Central ideas:
- Utility Computing
- SOA - Service Oriented Architecture
- SLA - Service Level Agreement

Enabling techniques:
- Virtualization
- Parallelized and distributed computing
- Service-oriented computing
- Autonomic Computing

- **scalability → elasticity**
  - ability → dynamic resource mechanism
- **availability → reliability**
  - service property → fault-tolerance mechanism
- **manageability → interoperability**
  - ability to manage → ability to integrate heterogeneous systems
- **accessibility → portability**
  - ability to access/use → ability to move across environments
- **performance → optimization**
  - desired outcome → improvement mechanism

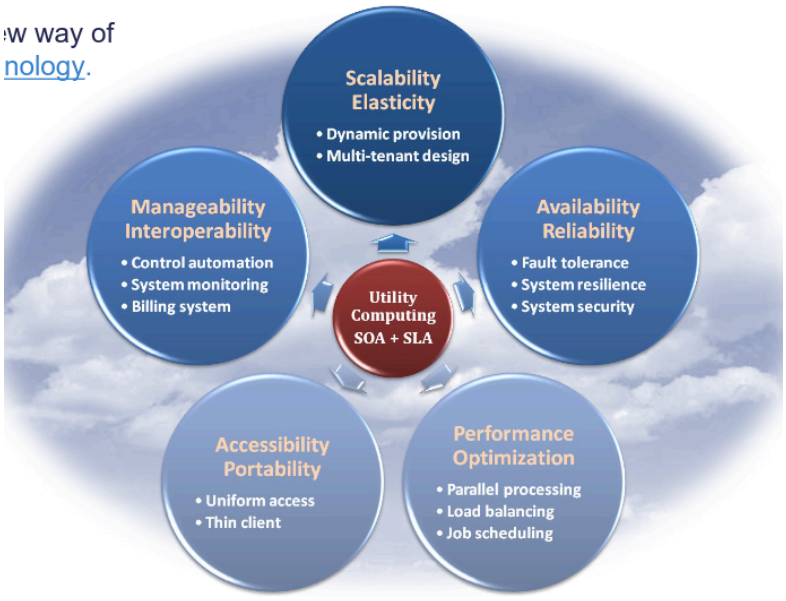


Figure 13: Cloud computing characteristics

4.1 Utility Computing

Definition 4.1 *(Utility Computing).* Utility Computing is a computing model in which computing resources, such as processing power, storage and networking are provided to users as metered services, similar to traditional public utilities like electricity, water, or gas.

Users do not own or manage the underlying hardware or infrastructure. Instead, they access computational resources on demand and pay only for the amount they actually consume.

Users do not care about **how** the work is done. Instead, they only concern about what they can get.

Users do not care about **what** the provider does. Instead, they only concern about the quality of service.

Users do not **own** the physical infrastructure. Instead, they only pay per usage

Users typically only care about the **quality of service**.

Utility computing is a model for **service provisioning**: the service provider makes **computing resources** and **infrastructure management available** to the customer as needed, and **charges them for specific usage** rather than a flat rate.

Like other types of on-demand utility, the utility model seeks to **maximize the efficient use of resources** (provider side) and/or **minimize associated costs** (provider and user).

4.1.1 Service oriented architecture (SOA)

Definition 4.2 *(Service Oriented Architecture).* Service Oriented Architecture (SOA) is essentially a collection of services which communicate with each other.

This type of architecture provides a loosely-coupled suite of services that can be used within multiple business domains. Usually it is implemented using the Web Service model.

Definition 4.3 *(Quality Of Service).* **QoS** (in the cloud context) denotes the **levels of performance**, **reliability**, and **availability** offered by the platform or infrastructure that hosts a reference application.

Definition 4.4 *(Service Level Agreement).* **A service-level agreement (SLA)** is a **contract between a service provider and a customer** that specifies, usually in measurable terms (QoS), what level of service the provider will furnish. Defines the **non-functional characteristics** guaranteed by the service. An SLA includes several **SLOs (Service Level Objectives)** that define the quality of service to be guaranteed through specific metrics.

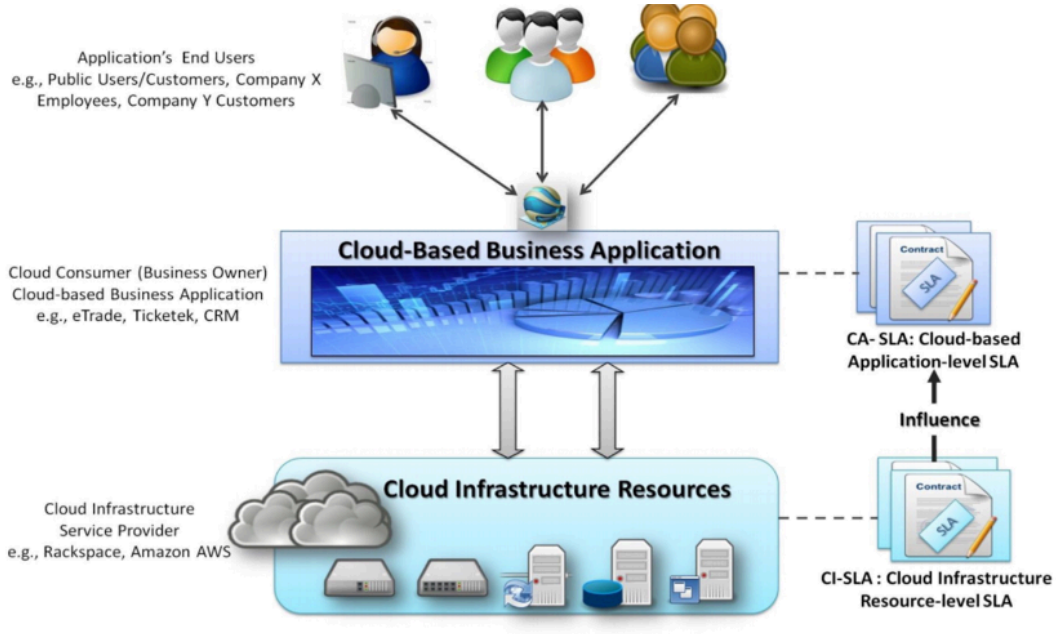


Figure 14: Service Level Agreement (SLA) and cloud infrastructure.

4.2 Scalability and Elasticity

- **Scalability** is a property of a system, a network, or a process, indicates its ability to either handle growing amounts of work in a graceful manner or to be readily enlarged (empowered).

- **Elasticity** is the ability to enforce adaptive "autonomic" actions to enable scalability.

These properties are achieved by **scaling**:

vertical (up or down):
- More powerful computing resources or bigger servers
- Single point of failure
- Human intervention
- Reasonable management overhead

horizontal (out or in):
- More computing resources (e.g., servers)
- Reliable – fail-over scenario (there is at least one node to take over)
- It can be fully automated (especially in the cloud)
- Growing management complexity

Definition 4.5 *(Dynamic provisioning).* Dynamic Provisioning describes a complex networked computing environment where **computing instances** are **provisioned** or **deployed** from an administrative console or client application.

The system might enable rules of automatizing the provisioning. In traditional computing model, two common problems:
- **Loss revenue / Loss Users** because of under provision (underestimation of system utilization).
- Organizations **overestimate** system utilization to ensure they can handle peak loads.

4.2.1 Traditional provisioning

Traditional provisioning involves allocating resources based on peak capacity requirements. Organizations **overestimate** system utilization to ensure they can handle peak loads. This conservative approach results in:
- Low average utilization: Resources sit idle during non-peak periods
- Higher operational costs: Paying for capacity that isn't being used most of the time
- Capital inefficiency: Significant investment in infrastructure that delivers poor return on investment.

This traditional model contrasts with modern cloud computing approaches, where resources can scale dynamically to match demand, reducing waste and improving cost efficiency.

4.2.2 Dynamic provisioning

Cloud resources can be be provisioned dynamically in response to actual demand patterns (Elasticity):
- **Meet seasonal demand variations**: Handle predictable cycles like holiday shopping periods or tax season.
- **Meet demand variations between different industries**: Adapt to sector-specific patterns and business cycles.
- **Meet burst demand for some extraordinary events**: Scale rapidly for unexpected spikes like viral content or flash sales.

With dynamic provisioning:
- Capacity (red dashed line) now closely follows the demand curve
- Resources scale up and down automatically to match actual needs
- The gray shaded area (waste) is dramatically reduced
- Capacity and demand are nearly aligned throughout the entire cycle

Organizations that do provisioning **without using cloud technologies** must commit to fixed capacity investments based on imperfect forecasts, inevitably leading to periods of both over-provisioning (waste) and under-provisioning (service issues). This inflexibility makes it nearly impossible to optimize costs while maintaining adequate performance.

Cloud provisioning creates near-perfect alignment between capacity and demand throughout the entire lifecycle, eliminating both waste and under-provisioning while converting fixed capital costs into variable operational expenses. This methodology gives us:

- No under-supply
- Reduction of over-supply
- Reduction of initial investments
- Improved Performances

4.3 Availability and Reliability

- **availability**:
  - is the proportion of time a system is in a functioning condition expressed as 100% minus unavailability
  - Cloud infrastructure usually provides high availability Ex. “Five Nines” system would statistically provide 99.999% availability
- **reliability**:
  - the ability of a system to perform its required functions under stated conditions for a specified period of time.

How to achieve these properties (at application level)?

- Fault tolerance system
- System resilience
- Reliable system security

4.3.1 Fault tolerance

Fault-tolerance is the property that enables a system to **continue operating properly** in the event of the **failure** of some of its components. If the operating quality decreases at all, the decrease must be proportional to the severity of the failure, as compared to a naively-designed system in which even a small failure can cause total breakdown.

Four basic actions:

- No **single point of failure**
  - One of the possible flaws in designing an application system
  - A part of a system which, if it fails, will stop the entire system from working
  - Preventive identification of the critical components of a system that would provoke a total systems failure in case of malfunction.
  - Can be prevented by **replication** of the critical components
- **Fault detection** of the failing component
  - **Fault Detection and Isolation (FDI)**: A subfield of control engineering which concerns itself with monitoring a system, identifying when a fault has occurred and pinpoint the type of fault and its location.
- **Fault containment** to prevent propagation of the failure
  - When a failure occurs, the system must be able to isolate the failure to the offending component.
  - To avoid other components to be taken down by the erratic or unexpected behavior
  - In the cloud usually faulty components are identified, isolated and terminated automatically
- Availability of **reversion** modes

Actions are implemented at Cloud level but the Cloud can help the designer to implement them at application level.

4.3.2 System resilience

Resilience is the ability to provide and maintain an **acceptable level of service** in the face of faults and challenges to normal operation. Resiliency pertains to the system's ability to **return to its original state** after encountering troubles. If a failure event knocks a system offline, a highly resilient system will return back to work and function as planned as soon as possible.

> **Definition 4.6** *(Disaster Recovery).* Disaster recovery is the set of **policies** and **procedures** related to be **prepared** for and **execute** the recovery after a disaster.

Some common strategies involve the use of backups of the whole system to an off-site location done periodically and the preparation of local mirrored systems, surge protection and UPS (Uninterruptible Power Supply) systems to ensure that the system can continue to operate in the event of a power outage.

4.3.3 System Security

Cloud security is a sub-domain of computer security, network security, and, more broadly, information security. It refers to a broad set of **policies**, **technologies**, and controls deployed to **protect data**, **applications**, and the associated **infrastructure** of cloud computing.

Important security and privacy issues :

- **Data Protection**
  - To be considered protected, data from one customer must be properly segregated from that of another.
- **Identity Management**
  - Every enterprise will have its own identity management system to control access to information and computing resources.
- **Application Security**
  - Cloud providers should ensure that applications available as a service via the cloud are secure.
- **Privacy**
  - Providers ensure that all critical data are masked and
  - that only authorized users have access to data in its entirety.

4.4 Manageability & Interoperability

Manageability:

- The ability of a system to be controlled either by means of self-control or by providing certain techniques to ease external controls
- Enterprise-wide administration of cloud computing systems.

Interoperability:

Interoperability is a property of a product or system to work with other products or systems, present or future, without any restricted access or implementation.

- Open interfaces / open access
- Web based interfaces
- Virtualization

These properties are achievable by system state monitoring and control automation.

4.4.1 System Monitoring

A System Monitor is a process within a distributed system for collecting and storing state data.

- Physical and virtual hardware state
- Resource performance metrics
- Network access patterns
- System logs

- ... etc

4.4.2 Billing System

Users pay per the **resources they used**. Cloud providers must

- determine the list of **service usage price**
- **record** the resource or service usage of each user, and then charge users by these records.
  - Get those information by means of monitoring system.
  - Automatically calculate the total amount of money which user should pay.
  - Automatically request money from use's banking account.

4.5 Performance optimization

This can be achieved by means of **load balancing**.

> **Definition 4.7** *(Load balancing).* Load balancing is a technique to **distribute workload evenly** across two or more computers, network links, CPUs, hard drives, or other resources, in order to get **optimal resource utilization**, maximize throughput, minimize response time, and **avoid overload**.

4.5.1 Economical aspects of cloud computing

Some aspects of cloud computing to be considered in an economic assessment:

- Model pay-as-you-go: no upfront costs for the cloud user - pay only for what you consume:
  - Form Capital Expenditure (**CAPEX**)
  - To Operational Expenditure (**OPEX**)

Elasticity gives risk transfer:

- No risks associated with incorrect sizing of the necessary infrastructure
- Provider absorbs infrastructure risk

Cloud give both the provider and the user the benefit of **economies of scale**:

- **Infrastructure Efficiency**: Large data centers achieve lower per-unit costs for compute, storage, networking,

electricity, cooling, and physical space

- **Cost Advantages**: providers pass savings to users through competitive pricing that small/medium organizations cannot match independently
- **Expertise Pooling**: Access to world-class skills, best practices, and cutting-edge technologies without building internal capabilities

5 Virtualization

First, we review the architecture of a computer, which is the basis of virtualization technology.

5.1 Computer architecture

Since the 1940's, computers have 5 classic components

- **Input** devices: Keyboard, mouse, cartridge, DVD ...
- **Output** devices: Display, printer ...
- **Storage** devices
  - Volatile **memory**: RAM (Random Access Memory)
  - Permanent **storage**: Magnetic, Optical, and Flash disks, ...
- Datapath, Control unit } Together, they are called the **Processor**

In the new concept of a computer, the **network** also is a new component.

The Central Processing Unit (**CPU**) loads programs from the Storage to the RAM and executes in **linear fashion** a succession of instructions (now you can call the programs processes). The "words" that the CPU understands belong to an **Instruction Set**.

An **Operating System** is a special program that controls the hardware and manages the **life-cycle of processes**. Several processes can run concurrently on the same HW, which is shared thanks to an OS that implements CPU time-sharing (**Multitasking**).

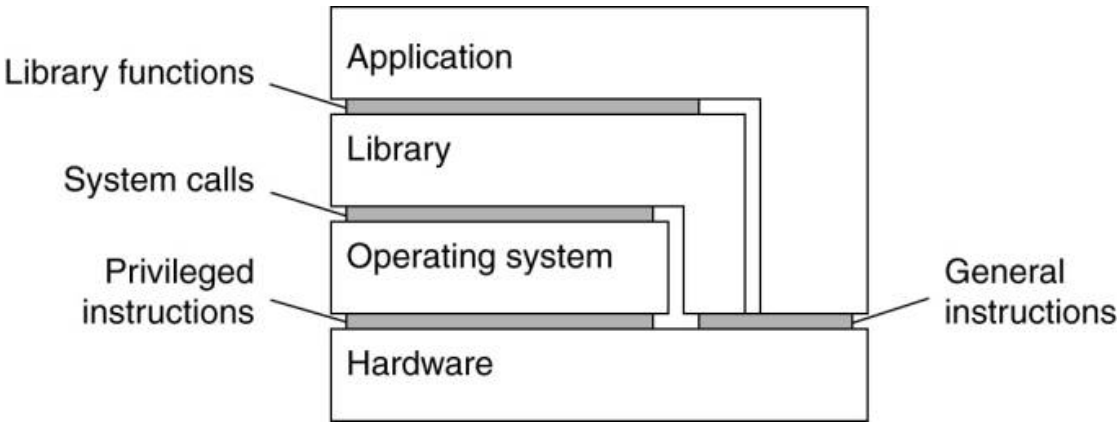


Figure 15: Layers of instructions and interaction with the hardware

We divide instructions by the level of abstraction:

- **Unprivileged machine instructions** (General instructions): an interface to the bare hardware that is available to any program
- **Privileged instructions**: provide an interface to the hardware for the OS
- **System calls**: provide an interface to the operating system for applications
- **APIs**: high-level programming endpoints that may exploits OS system calls

5.1.1 Emulation

Emulation is a technique where one computer system (the **host**) **mimics the hardware and instruction set** of a different computer system (the **guest**). It **translates** instructions from the guest architecture into instructions the host can execute, **recreating the complete behavior of the original system in software**.

5.2 Virtualization and Virtual Machines

Virtualization is a technology that creates **abstracted, logical instances of physical computing resources**. It uses a **hypervisor** (or Virtual Machine Monitor) to partition physical hardware and create multiple isolated virtual environments called **Virtual Machines** (VMs).

Each VM runs its own operating system and applications, believing it has dedicated hardware, while sharing the underlying physical resources, i.e., the ISA (Instruction Set Architecture).

Aspect	**Virtualization**	**Emulation**
Architecture	Same ISA (e.g., x86 on x86)	Different ISA (e.g., ARM on x86)
Performance	Near-native speed	Significantly slower
Translation	Minimal/no instruction translation	Full instruction translation required
Use case	Server consolidation, cloud computing	Legacy system preservation, cross platform testing
Example	VMware ESXi, KVM, Hyper-V	QEMU (in full emulation mode), DOSBox, game console emulators

Definition 5.1 *(Virtual machine).* A **virtual machine** provides a complete **virtualized hardware platform** to support running a complete operating system. The resources of the underlying physical machine can be shared among several virtual machines, each with its own operating system.

Virtual machines are created and managed by a **Hypervisor** (a.k.a. Virtual Machine Monitor) Examples: VirtualBox, KVM, VMware and Xen.

We can further distinguish between two types of virtual machines:

- **System virtual machines**: Provide a complete system platform that supports the execution of a full operating system (e.g., VMware, VirtualBox, KVM).
- **Process virtual machines** (also application virtual machines or managed runtime environments): Designed to run a single program, providing a platform-independent programming environment (e.g., Java Virtual Machine, .NET Common Language Runtime). They only virtualize the resources needed to ecevute a specific application.
  - **Process-level abstraction**: Virtualizes at the application layer, not the hardware layer
  - **Single application**: Each VM instance runs one application or process
  - **Platform independence**: "Write once, run anywhere" - code runs on any platform with the VM
  - **Managed execution**: Provides runtime services like memory management, garbage collection, and security
  - **Intermediate representation**: Applications are typically compiled to platform-independent bytecode, which the VM interprets or JIT-compiles

From here on, we will focus on system virtual machines, which are the most relevant for cloud computing.

Definition 5.2 *(Hypervisor).* An **Hypervisor** (or Virtual Machine Monitor) is a software that creates and manages virtual machines by abstracting and partitioning physical hardware resources. It leverages hardware virtualization features (such as Intel VT-x or AMD-V) provided by modern CPUs to efficiently manage virtual machines. Shares the available resources in such a way that each VM sees them as **dedicated**.

5.2.1 Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege).

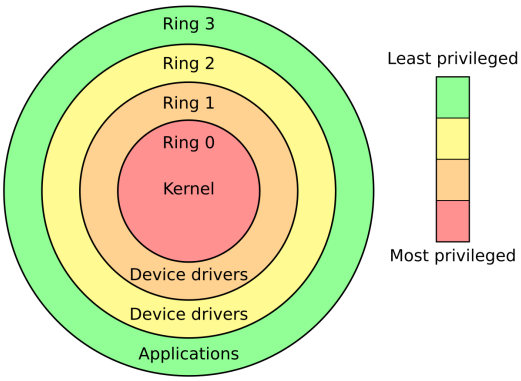


Figure 16: Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege). Operating Systems kernels expect to run at Ring 0 to execute privileged instructions directly on hardware. Hypervisors need to intercept these instructions: early solutions used:

- trap-and-emulate.

> In a virtualized environment, the guest OS believes it has full control over the hardware, but it actually runs in a deprivileged mode (typically user mode or a less privileged ring). When the guest OS attempts to execute a privileged instruction, the CPU generates a trap (exception), transferring control to the hypervisor, which then emulates the instruction's behavior in a safe, virtualized manner.

- binary translation.

> Binary translation is a virtualization technique that dynamically scans, analyzes, and rewrites guest code at runtime to replace problematic instructions with safe equivalents that can be executed in a virtualized environment. It was the solution VMware developed to virtualize x86 architecture before hardware virtualization support existed.

Hardware-assisted virtualization (Intel VT-x, AMD-V) introduced new CPU modes that allow guest OSes to run as if they were in Ring 0, while still being controlled by the hypervisor. CPUs add a new mode that lets the **hypervisor run in "root mode"** while the **guest OS runs in "non-root mode"**. When the guest tries to execute a privileged instruction, the CPU automatically traps it and hands control to the hypervisor (Privileged instruction handling), but also:

- Efficient context switching between guest and host environments.
- Memory virtualization support
- I/O virtualization technologies like Intel VT-d and AMD IOMMU allow safe and efficient device sharing between VMs.

5.3 Virtualization of resources

Virtualization can be applied to various resources, here we show the most relevant for cloud computing.

5.3.1 Memory virtualization

Memory management is a critical challenge in virtualization: guest OS maintains its own page tables assuming it owns physical memory.

> **Definition 5.3** ***(Shadow page tables)***. Shadow Page Tables are maintained by the hypervisor, which keeps synchronized copies of guest page tables, translating guest-virtual to host-physical addresses (but with high overhead).

Definition 5.4 ***(Hardware-assisted page tables).*** Extended Page Tables (EPT, Intel) or Nested Page Tables (NPT, AMD) add a second level of hardware translation, reducing overhead.

These technologies drastically improved performance, enabling scalable virtualization in cloud data centers.

5.3.2 I/O virtualization

Input/Output devices must also be virtualized for guest systems. We have difference approaches:

- **Device emulation**: hypervisor emulates hardware devices, highly portable but slow.
- **Paravirtualized drivers** (e.g. VirtIO): guest OS communicates directly with hypervisor through optimized interfaces, achieving better performance.
- **Direct device assignment with RS-IOV** (single root I/O virtualization): near-native performance by partitioning PCI (Peripheral Component Interconnect) devices, at the cost of flexibility and migration limitations.

5.3.3 Network virtualization

Network virtualization abstracts physical networking resources (switches, routers, links) into Software-Defined logical Networks (SDN). It allows multiple isolated virtual networks to coexist on the same physical infrastructure, enabling multi-tenancy in cloud data centers. **Virtual switches** (Linux bridge, Open vSwitch) **connect VMs or containers within each host** as if they were plugged into a real switch. **Overlay networks** (e.g., VXLAN, GRE tunnels) interconnect these virtual switches **across hosts, creating logical topologies independent of the underlying physical network**.

The benefits of network virtualization include:

- strong isolation
- programmability through SDN controllers
- elastic provisioning
- simplified management of large-scale infrastructures.

5.4 Advantages of virtualization

- **Unified vision of physical resources**
  - resources can be on different physical machines or on the network
- **Centralized management** of hardware and software
- **Workload isolation**. Programs are executed inside the VM and are isolated w.r.t. to the host or other VMs
  - Security
- **Consolidation** of virtualized items
  - Optimization of physical resource usage
- **Easy migration** of virtualized items
- Easy way to implement **Redundancy**.
  - Virtualized items can be easily copied

5.4.1 Workload isolation

Through virtualization, workload isolation is achieved since **all program instructions are fully confined inside a VM**, which leads to improvements in security.

Reliability achieved because software failures inside one VM do not affect others.

Performance control is attained since execution of one VM should not affect the performance of another VM.

5.4.2 Workload migration

Workload migration, a.k.a. **application mobility**, improves:

- hardware maintenance,
- performance,
- fault tolerance.

It is done by allowing the guest OS within a VM to be **suspended, migrated to a different platform**, and **resumed immediately** or **preserved** to be restored at a later date. A VM's state includes a full disk, configuration files, and an image of its memory (RAM).

5.4.3 Consolidation

Virtualization makes it possible to consolidate several virtual machines onto a single physical system, reducing the **total cost of ownership** (TCO), which is an estimate of all the direct and indirect costs involved in acquiring and operating a product or system over its lifetime.

5.5 Taxonomy of hypervisors

We can divide hypervisors into two main categories:

- **Bare-metal hypervisors** (Type 1):
  - run directly on the host's hardware to control the hardware and manage guest operating systems. Examples: VMware ESXi, Microsoft Hyper-V, Xen.
  - The hypervisor communicates directly with system hardware rather than relying on a host operating system.
  - The hypervisor is basically a lightweight OS.
  - **advantages**:
    - Improved I/O performance.
  - **Disadvantages**:
    - More complex to set up and manage.
    - It depends on the hardware platform.
- **Hosted hypervisors** (Type 2):
  - run on a conventional operating system just as other computer programs do. Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop
  - It allows users to run various guest operating systems side by side to their own applications.
  - **Advantages**:
    - Ease of installation and configuration
    - Unmodified Host OS & Guest OS
    - Run on a wide variety of computers
  - **Disadvantages**:
    - Performance overhead due to the additional layer between the hardware and the virtual machines.

5.6 Virtualization techniques

There are several virtualization techniques available:

- **Software Virtualization**
- **Hardware Assisted Virtualization**
  - The hardware implements specific technologies to support virtualization
  - Full virtualization and Para virtualization can exploit it for performance

Here we focus on software virtualization techniques.

5.6.1 Full Virtualization using binary translation

Provide a virtual machine environment that is a full emulation of the underlying hardware.

- The guest OS does not need to be modified

- Guest OS is not aware the existence of the virtual environment
- Each VM is independent of each other

The hypervisor executes **privileged instructions** by dynamically translating the instruction of guest OS into a sequence of instruction appropriate to execute in real hardware (**Binary Translation**). Unprivileged instructions are executed directly.

5.6.2 Para Virtualization (a.k.a. OS assisted Virtualization)

The guest OS is modified to be aware that it is running in a virtualized environment. Instead of trying to **fully emulate hardware** (as in full virtualization), the guest OS uses **special calls** to interact directly with the hypervisor. These calls are usually called **hypercalls**.

- the guest os are aware of the hypervisor and use special calls to interact with it
- performance boost
- seamless integration (mouse, keyboard, screen)

paravirtualized drivers are optimized, virtualization- aware device drivers that replace slow device emulation with direct, efficient communication between the guest OS and the hypervisor. Example: **VirtIO** (KVM/QEMU): standard for paravirtual I/O devices (network, block storage, etc.).

5.6.3 OS-Level virtualization (Containerization)

The host operating system itself provides multiple isolated execution environments, known as **containers** (or zones, jails, etc.), rather than emulating full hardware machines. Unlike hypervisor-based virtualization, OS-level virtualization shares the **same kernel between all containers**. Each container runs as a set of isolated processes, with its own view of the filesystem, process table, network stack, and resource quotas. **The virtualization capability is provided by the host OS**, which performs all the functions of a fully virtualized hypervisor

- Linux Container (Docker, LXD…)

6 deep dive: IaaS (AWS)

Amazon AWS is a set of building-block services, designed to work independently, that can work well together:

- They share a common naming convention and authentication
- Minimize internal connections and dependencies

Every activity in AWS can be accessed by making a **web service call**.

6.1 Key concepts

Key concepts:

- EC2 = **Compute**: runs the software.
- S3 = **Object storage**: stores objects accessible through APIs.
- EBS = **Block storage**: provides a disk for the virtual machine.

6.1.1 S3

Definition 6.1 *(Object store).* An **object store** is a storage architecture that manages and stores data as objects, rather than as files in a file system or blocks in a traditional storage system. Each object in an object store includes three components:

1. **Data**: The actual content you are storing, such as a file, video, or image.
2. **Metadata**: Descriptive information about the object, like its name, creation date, and other relevant details.
3. **Unique Identifier**: A unique ID or key assigned to each object, which allows it to be retrieved without needing to know the location of the data (e.g., file path).

Simple Storage Service (S3) is an **object storage** solution:

- with web interface (REST, SOAP, BitTorrent)
- Unlike file systems with hierarchical directories, object stores use a flat address space where every object is stored at the same level.
- Objects up to 5 Terabyte each
- Data automatically distributed across a minimum of three AZs(Availability)
- Automatic data replication (Durability)
- Can be queried using SQL or processed by Big Data solutions

SLA: 99.99% monthly uptime, no more than 4 minutes per month.

Useful for **Backup** & **Recovery**, **Archiving**, and **Disaster Recovery**.

6.1.2 EBS

Definition 6.2 *(Block store).* A **block store** is a storage architecture where data is divided into fixed-size chunks called "blocks." Each block is identified by an address within the storage volume. The operating system or application can read and write individual blocks, while a file system typically organizes them to store and retrieve files. Block storage is commonly used in traditional storage devices, such as hard drives and solid-state drives (SSD), as well as in cloud storage systems.

Elastic Block Store (EBS) Volume:

- An addressable disk volume that can be attached to any running instance (a.k.a. VM) in the same Availability Zone
- HDD and SDD
- The volume can then be formatted, mounted, and used as if it were a local disk drive
- Volumes have a lifetime independent of any particular instance (VM)

- A volume can persist even when no instances are running
- Price from $0.025 GB-Month to $0.124 GB-Month (2008)

6.1.3 Security group

A **security group** defines the allowable set of inbound and outbound network connections for an EC2 instance (VM). Each group is named and consists of a list of protocols, ports, and IP address ranges. A group can be applied to multiple instances, and a single instance can be regulated by multiple groups.

A Security Group acts as a **virtual firewall** for an EC2 instance.

- **Rules**: Specify which network traffic is allowed, based on protocol (e.g., TCP), port (e.g., 443 for HTTPS), and source IP address.
- **Inbound and outbound traffic**: Rules control both incoming and outgoing connections.
- **Flexibility**: A security group can be associated with multiple instances, and an instance can have multiple security groups.

> Security groups are stateful: response traffic for an allowed connection is automatically permitted.

6.2 EC2

EC2 implements the **programmable data center metaphor**, later it will be called **Infrastructure as a code (IaC)**. You can get access to multiple Availability Zones in multiple geographic regions and provision your virtualized data center on demand. Servers, operating system images, firewall entries, load balancers, IP addresses, and disk storage volumes are all available in a matter of seconds, by simply making a web service request "to the cloud".

> **Definition 6.3** *(Amazon Machine Image (AMI)).* Contains the operating system and can also include additional software and layers of your application such as database servers, middleware, web servers,...

- Prebuilt AMIs can be started.
- AMIs can be customized, shared, or even sold.
- Each AMI has a unique ID (e.g., ami-bf5eb9d6)

> **Definition 6.4** *(Instance).* Represents one running copy of an AMI. Any number of copies of the same AMI can be launched.

AWS allows you to allocate static IP addresses and then attach them to your instances. Each instance can have at most one such address attached. When the one instance fails, the elastic IP address is remapped to another VM (if it belong to the same group).

An instance can be running any one of several operating systems, including Linux and Windows Server. When you start up an Amazon EC2 instance you must specify the **instance type**. Modern EC2 instances now list specific details like:

- Number of vCPUs
- Actual processor models (e.g., Intel Xeon, AMD EPYC, AWS Graviton)
- Clock speeds
- Physical cores vs. threads

6.2.1 Ephemeral resources

Ephemeral resources are components that only exist while an instance is running and are lost when the instance stops or terminates. They're not permanent: the data vanishes when the instance goes away.

> EC2 instances are ephemeral!

In case of failures, stored data and state information is generally lost.

For production workloads, implement:

- **Redundancy**: Multiple instances across availability zones
- **Failover**: Automatic instance replacement
- **Recovery**: Regular backups and automation scripts

Best suited for **stateless** application patterns

> The **local disk storage** included with each EC2 instance is ephemeral

The storage will remain intact if a running instance is rebooted, but data is permanently lost when instance terminates. Use persistent storage instead:

- Amazon EBS volumes for block storage
- Amazon S3 for object storage
- Amazon EFS for shared file systems

6.2.2 Persistent Resources

Once allocated to your account, they are expected to remain operational in the face of transient or permanent hardware or software failures. The Amazon implementation of a persistent resource makes uses of **redundancy**, **automated failover**, and **automatic recovery** to provide stable resources. The following EC2 resources are persistent:

- Elastic IP addresses
- EBS volumes
- Security groups
- Elastic Load Balancers
- AMIs stored in S3 or as snapshots of EBS volumes

6.2.3 Pricing model

The pricing model determines how compute capacity is paid for and what flexibility you retain.

$$\text{Cost} = \text{compute usage} + \text{related resources and data transfer}$$

There is no single EC2 price: the estimate depends on the workload, configuration, Region and usage pattern. The **configuration** depends on:

- Instance family and size
- Operating system / software
- Region and tenancy

Additional costs may be incurred for:

- EBS storage and performance
- Data transfer (inbound is free, outbound is charged)
- Public IP addresses
- Monitoring and other services

6.2.3.1 Purchase options

On-Demand:

- Pay for the compute you use; no long-term commitment.
- Best for:
  - learning
  - development
  - variable or uncertain demand.

Reserved Instances

- A 1 or 3 year pricing commitment for a defined EC2 configuration/usage pattern
- availability and flexibility depend on the RI type.

Savings Plans

- Commit to a consistent amount of eligible compute usage for 1 or 3 years in exchange for lower rates.
- Best for: predictable baseline usage with flexibility.

Spot Instances

- Use spare EC2 capacity at a variable discount; AWS can interrupt it when capacity is needed.
- Best for: fault-tolerant batch jobs and workloads that can resume.

Workload pattern	Typical starting point	Why it fits	Main trade-off
New / uncertain	On-Demand	No long-term commitment	Higher unit price than commitment options
Stable baseline	Savings Plans; compare RIs	Discount in exchange for commitment	Commitment may outlast demand
Flexible batch work	Spot	Can reduce compute cost	Capacity can be interrupted
Need capacity guaranteed	Capacity Reservation + suitable pricing	Capacity held in a specified Availability Zone	Reservation is a separate concept from discount

> **Rule of thumb**:
> - flexibility → On-Demand
> - steady commitment → Savings Plans / RIs
> - interruption-tolerant → Spot

Purchasing options and capacity reservations solve different problems; they can be combined.

To obtain current prices:

1. **Define the workload**
   - Choose Region and Availability Zone needs
   - Select instance family and size
   - Specify OS, tenancy and runtime
2. **Add related resources**
   - EBS volume size and type
   - Data transfer assumptions
   - Public IPv4 and monitoring
   - Other dependent service
3. **Compare scenarios**
   - On-Demand vs Savings Plans / RIs
   - Spot only if interruption is acceptable
   - Estimate monthly and annual costs

6.3 Autoscaling

In traditional monolithic deployments, the application's database, API, and business logic coexist on a single physical or virtual server. As the user base grows, the system's capacity to maintain a consistent Quality of Service (QoS) deteriorates. The only available scaling strategy in such a setup is **vertical scaling**, that is increasing the resources of a single machine (CPU, RAM, or storage). This approach introduces a **single point of failure** and quickly becomes

economically inefficient due to the **nonlinear** growth of hardware costs. This limitation paved the way for **cloud-based architectures**, where **scalability** and **resilience** can be achieved through **distributed resources** rather than hardware upgrades.

6.3.1 Layered architecture

In modern architectures, applications are organized in layers that separate **stateless components** (e.g., web servers, APIs) from **stateful** ones (e.g., databases, persistent storage).

This separation allows independent scaling strategies:

- Stateless layers can scale horizontally by replicating instances across multiple machines.
- Stateful layers require specialized handling to preserve data consistency and durability.

As user demand grows, the stateless layer typically faces CPU-bound scaling needs, while the persistence layer depends on I/O and storage optimization.

6.3.2 Load balancing

In horizontally scaled systems, a load balancer acts as the single entry point for all client requests, distributing traffic among multiple application instances. Its goal is to **optimize resource utilization**, **minimize response time**, and **ensure high availability**.

Common scheduling algorithms include:

- **Round Robin** (RR): sequentially forwards each request to the next available node.
- **Least Used** (LU): selects the node with the lowest current load.

Modern cloud load balancers continuously perform **health checks** and cooperate with other cloud services to **dynamically adjust** the number of active instances in response to changing workloads.

6.3.3 Amazon Auto Scaling group

Usually, **multiple copies of a service run simultaneously** to serve the volume of customer traffic. End-users see only one interface for the application, but behind it many identical VMs (EC2 instances) are running, each handling a share of the incoming requests. Managing groups of instances can be cumbersome. EC2 provides **Auto Scaling groups** as a solution. Auto Scaling groups are defined with a minimum and maximum number of identical EC2 instances. Useful for controlling a fleet of EC2 instances that can handle a variable load. An auto scaling group is a collection of Amazon EC2 instances that:

- share the same characteristics (homogeneous virtual machines)
- are treated as one logical grouping for the purposes of scaling and management

Auto Scaling groups automatically:

- scale the number of virtual machine instances (horizontal scaling) or
- maintain a fixed number of instances (fail recovery – fleet management)

The EC2 instances can be from the same region or from one or more availability zones.

6.3.3.1 Launch Configuration

Every Auto Scaling group is associated with a launch configuration, which describes an instance (VM) of the group (parameters necessary to create the instances). Only one launch configuration can be attached to an Auto Scaling group at a time.

New or updated launch configuration can be introduced. Future new instances will be created and launched using the new configuration parameters, existing instances are not affected. When Auto Scaling needs to scale down, it first terminates instances that have an older launch configuration.

6.3.4 Auto Scaling with AWS

Auto Scaling is a web service designed to launch or terminate EC2 instances automatically based on:

1. User-defined policies
2. Schedules
3. Health checks

1 and 2 are useful to respond automatically to changing conditions. It is used to specify how Auto Scaling should respond to changes.

It guarantees that the number of EC2 instances running:

- Increases seamlessly during demand spikes
- Decreases automatically during demand lulls
- Stays the same in case of failures

The services involved in Auto Scaling are:

- **CloudWatch (CW)**:
  - Provides monitoring over EC2 (CPU load average, disk I/O rate, and network I/O rate)
- **Elastic Load Balancer (ELB)**:
  - Distributes traffic across any number of EC2 instances (in the same Availability Zone or across several zones in a Region)
  - Performs also periodic health checks on the instances of the group, and will stop sending traffic to unhealthy instances
  - Health information is then collected by CloudWatch
- **Auto Scaling**:
  - Uses the data collected by CW to build systems that can scale out (adding more instances) and scale in
  - (shutting down instances) over a defined auto scaling group
  - Usually responds to triggers on CPU/memory utilization and latency, implementing a policy

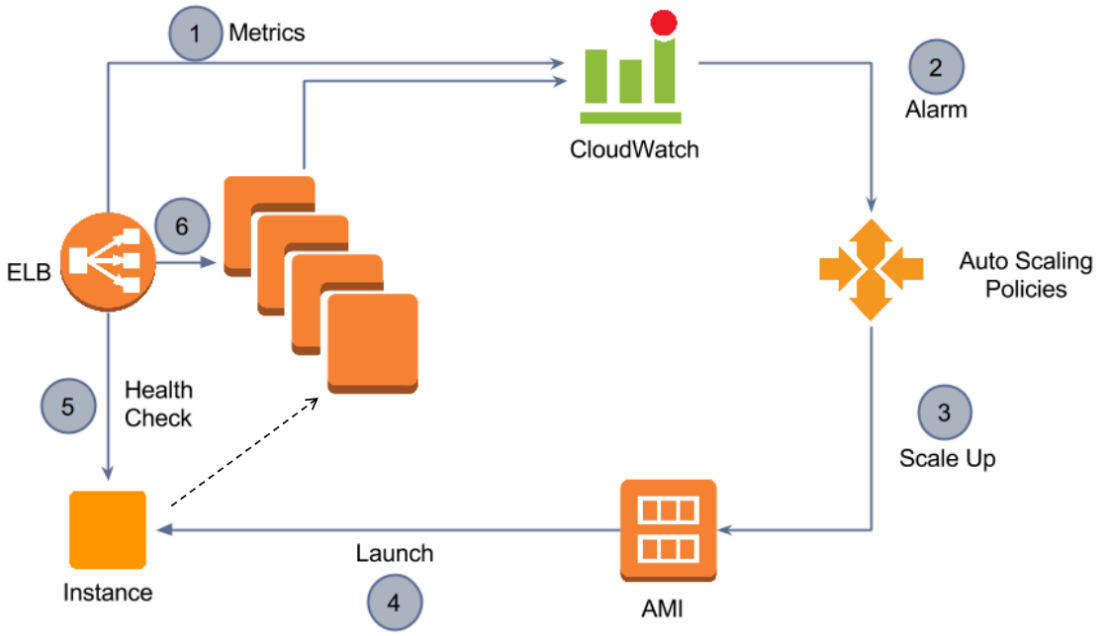


Figure 17: Auto Scaling pipeline

Auto Scaling attempts to distribute instances evenly between the Availability Zones that are enabled for an Auto Scaling group.

- Auto Scaling attempts to launch new instances in the Availability Zone with the fewest instances.

- If the attempt fails, however, Auto Scaling will attempt to launch in other zones until it succeeds

Certain operations and conditions can cause an Auto Scaling group to become unbalanced. Rebalancing activities can take place.

6.3.4.1 Health checks

A **health check** is a **periodic test** performed to assess the operational status of each instance within an Auto Scaling group. It typically involves invoking a predefined endpoint or using a built-in monitoring probe to verify whether the instance responds correctly and within acceptable performance thresholds. When an instance fails a health check, due to degraded performance or unresponsiveness, Auto Scaling **automatically terminates it and launches a replacement instance**. This mechanism ensures that the group remains consistent, available, and self-healing, without manual intervention.

6.3.4.2 Triggers in Auto Scaling Systems

A trigger defines the condition under which an Auto Scaling action is executed. It combines two core elements:

- **CloudWatch Alarm** — continuously monitors a chosen performance metric (e.g., CPU utilization, latency, or network throughput) and detects when it crosses a defined threshold.
  - An object that watches over a single metric. It can change state depending on the value of the metric
  - When an alarm changes state one or more actions are triggered
  - To create an alarm, specify:
    - The metric to watch
    - The threshold values for the metric
    - The number of evaluation periods
  - An alarm has three possible states:
    - OK: The metric is within the defined threshold
    - ALARM: The metric is outside of the defined threshold
    - INSUFFICIENT_DATA: The metric is not available, or there is not enough data available for the metric to set the alarm state to OK
  - An action is invoked when an alarm changes state and remains in that state for several time periods
- **Auto Scaling Policy** — specifies the corrective action to perform (e.g., launch or terminate instances) when the alarm changes state.
  - A policy defines the scaling action that an Auto Scaling group performs when a monitoring alarm changes state.
  - It specifies **how** and by **how much** capacity should change
  - AWS supports multiple policy types:
    - **Simple scaling**: single rule and fixed adjustment.
    - **Step scaling**: multiple thresholds with proportional actions.
      - adjustment depends on size of alarm breach
    - **Target tracking scaling**: continuous feedback around a target metric.
      - target value for a specific metric
    - **Scheduled scaling**: time-based capacity adjustments
      - Configure Time based Auto Scaling plan
      - Specify the date and time of execution, minimum, maximum,and desired instance size of capacity

When the **alarm threshold** is reached, the corresponding policy initiates a **scaling activity**.

6.3.4.3 Fleet management

Fleet management ensures that the instances in EC2 fleet are performing as designed. **ELB** and **CW** monitor the health of each EC2 instance launched. If any instance terminates unexpectedly:

- The termination is detected
- A replacement instance is launched

There are three main functions that Auto Scaling performs to automate fleet management for EC2 instances:

- Monitoring the health of running instances
- Automatically replacing unhealthy instances
- Balancing the traffic across Availability Zones

6.3.4.4 Cooldown — Stabilizing the Scaling Feedback Loop

A **cooldown period** is a configurable interval following a scaling activity during which Auto Scaling temporarily suspends additional scaling actions. This period allows the system to stabilize and for monitoring metrics (e.g., CPU utilization, latency) to reflect the impact of the previous capacity adjustment. Without a cooldown, consecutive scaling events could occur before the system reaches equilibrium, leading to **oscillations** (thrashing) and unnecessary resource churn.

7 deep dive: PaaS (Azure)

Let us start with a review of what PaaS is. PaaS is an integrated environment that supports the development, running and management of cloud- based applications

- Oriented to software development and operations with proper design tools, middleware and APIs
- Offering key services (data storage, caching, mail, ...)
- Typically supporting a set of programming languages (Python, Java, C#, Scala, ...)

The user can **deploy onto the cloud infrastructure** applications created. The user **does not manage or control the underlying cloud** but has **control over the deployed applications** and possibly application hosting environment configurations. This is possible thanks to runtime environments that abstract the underlying infrastructure and provide a set of services to the applications. Users use a programming IDE to develop their service among PaaS. This IDE should integrate the full functionalities which supported from the underling runtime environment. It should also provide some development tools, such as profiler, debugger and testing environment.

The traditional issues in software development are:

- Creation of a traditional development environment
- Acquiring servers
- Installing and configuring operating systems, runtime environments and databases
- Creating and configuring a source control repository (configuration management) like Git
- Move or copy existing code
- Deploy, test, and run the code that is developed

With PaaS developers do not need to care about installation, configuration and operation of servers, runtime and databases, and they can **focus on their development tasks** and on managing versions and configuration of the application.

They are supported by:

- **development services**, including build, test, and software repositories
- **run-time services**, including containers/sandboxes for running and scaling applications

They can test workload spikes by letting the application **self-acquiring new resources** (possibly, automatically released when they are no more needed). Different test workload patterns can be generated.

The **application life-cycle** can be made **"agile"** with continuous development and deployment

- Deployment is faster and safer
- The state of software becomes **more visible to the organization**
- **Collaboration** among members of the development team increases even in a dispersed/ global setting
- **Developers and testers can interact with the same hosted application** and runtime as the operators

Goals: make the workflow

- **Repeatable**
- **Reliable**
- **Predictable**
- **Fast (Low Cycle Time)**

7.1 Azure Web App service

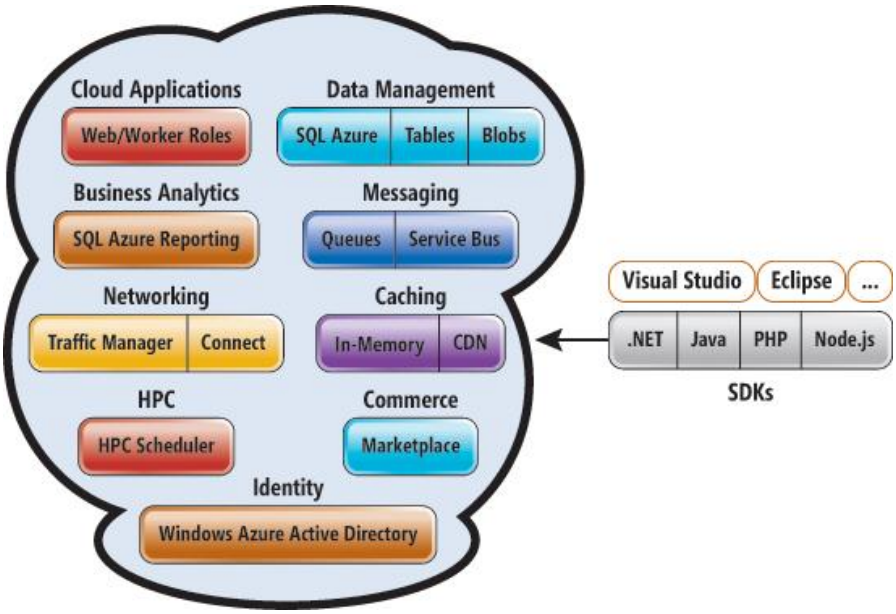


Figure 18: Azure PaaS services

Azure App Service provides a fully managed PaaS environment for deploying and running web applications. Applications execute inside isolated sandboxes, where developers focus on the application logic rather than managing servers or operating systems.

7.1.1 Roles

A PaaS application is composed of one or more **roles**, which are the building blocks of the application. Execution roles:

- **Web roles** handle user-facing logic and process HTTP(S) requests.
- **Worker roles** for resource intensive workloads (back-end)

Both roles run inside **Azure Cloud Services**, which are composed of one or more VMs. However, these VMs are **not directly exposed to the user** — they are **abstracted and orchestrated** by the platform to behave as PaaS components.

- You **do not manage** the VM's operating system, networking, or runtime configuration, **Azure takes care of deployment, patching, scaling, fault recovery, and load balancing**.
- What you deploy is **your role package**: an application bundle containing your code, configuration, security, and any required startup tasks. Azure's fabric controller then places it into a VM instance.
- The VM can be **recycled or replaced** by the platform at any time (for updates or scaling), which is why **roles must be stateless** and store persistent data externally.

7.1.1.1 Web role

A **Web Role** is a cloud-hosted execution container within Azure App Service specifically designed to run web applications and expose HTTP endpoints. It operates within a **managed IIS (Internet Information Services)** environment preconfigured by Azure, allowing developers to deploy web applications without provisioning or maintaining web servers manually. Each Web Role instance:

- Runs in its own **sandboxed virtual machine**, managed by Azure.
- Can host **multiple web applications** or services built with technologies such as ASP.NET, Java, Python, PHP, or Node.js.

- Is automatically integrated with Azure's **load balancer**, distributing incoming traffic across multiple instances to ensure availability and scalability.
- Can be **scaled out** dynamically based on demand (e.g., CPU usage or request rate).
- Is **stateless** by design — application state should be externalized (e.g., in Azure Storage, Redis, or SQL Database).

The Web Role represents the **frontend tier of a PaaS application**—an automatically managed web-hosting environment that **abstracts away infrastructure complexity** while maintaining scalability and resilience.

7.1.1.2 Worker Role

A **Worker Role** is a **background processing component** within Azure App Service that executes **asynchronous or long-running workloads** decoupled from direct user interaction. Each Worker Role instance:

- Executes in an **isolated, managed virtual virtual machine** provisioned and monitored by Azure.
- Is typically **connected to a queue or service bus**, receiving tasks generated by Web Roles or other services.
- Handles **resource-intensive operations**, such as data transformation, batch computation, or integration tasks.
- Can be **scaled independently** of Web Roles, enabling flexible workload distribution and cost optimization.
- Maintains **stateless behavior** — state persistence should rely on external storage services (e.g., Azure Blob, Table, or SQL Database).

The Worker Role acts as the **backend execution tier** of a PaaS application—processing queued or event-driven workloads while **Azure manages infrastructure, scaling, and availability**.

7.2 Azure Auto scaling Application Block

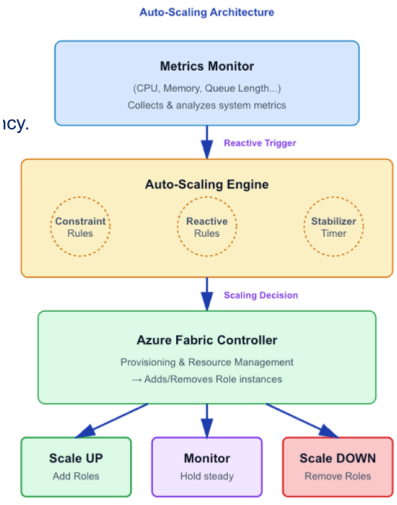
Azure's **Auto Scaling Application Block** is a component that enables automatic adjustment of deployed instances based on **defined scaling policies**. It allows applications to **scale out** (add instances) or **scale in** (remove instances) in response to workload changes, ensuring both performance stability and cost efficiency.

Types of scaling rules:

- **Constraint rules**: useful to define lower or upper bounds on the number of instances of a certain role
- **Reactive rules**: dynamically trigger scaling actions in response to monitored metrics.

Related mechanisms:

- **Scaling Groups**: allow multiple roles to share common scaling logic and thresholds.
- **Throttling Actions**: temporarily limit workload intensity when scaling cannot keep up.
- **Stabilizer (Cooldown Period)**: prevents frequent up/down oscillations by enforcing a minimum time (typically 20 minutes) between scaling actions.



7.2.1 Throttling

Throttling is a **protective mechanism** that limits the rate at which clients or workloads can consume resources when system capacity is under stress. Throttling ensures **the system remains stable and continues to serve legitimate requests**, while preventing overload or resource exhaustion.

Key concepts of throttling:

- **Rate limits / quotas** — define a maximum number of allowed operations per time window (e.g. requests per second, calls per minute).
- **Rejecting or delaying excess requests** — once limits are hit, further requests may be refused (e.g. HTTP 429) or queued/delayed.
- **Graceful degradation** — nonessential operations may be slowed or disabled to preserve core functionality.
- **Combining with auto-scaling** — throttling can act as a buffer until scaling completes; once new capacity is available, normal rates resume.
- **Per-tenant / per-user limits** — in a multi-tenant system, throttling prevents one user from monopolizing shared resources.

7.3 Storage for persistent data

There are several options for storing persistent data in Azure PaaS applications, depending on the nature of the data and access patterns:

- **Blobs (Binary Large Object)**
  - Provide a simple interface for storing named files along with metadata for the file
  - Storage for any type of entity such as binary files and text documents
  - Is a **Distributed File Service (DFS)**
    - Scalability and High availability
    - BLOB file is distributed between multiple server and replicated at least 3 times
    - Only one level of folders is possible (called containers)
  - Access to BLOBs is provided through a **REST API**, which allows for operations such as uploading, downloading, and managing files.
- **Tables**
  - Provide lightly structured storage with a set of entities that contain a set of properties
  - Not RDBMS
    - No relationships between entities
    - NoSql – key-value with flexible schema
  - Entity can have up to 255 properties - Up to 1MB per entity
  - Mandatory Properties for every entity
    - PartitionKey & RowKey (the only indexed properties)
      - Uniquely identifies an entity
      - Same RowKey can be used in different PartitionKey
      - Defines the sort order
    - Timestamp
  - Strongly consistent (every reader see the latest version of data)
  - Supported by the following frameworks:
    - .NET, Java, Android, C++, Node.js, PHP, Ruby, and Python
- **Queues**
  - Provide reliable storage and delivery of messages

7.3.1 Azure SQL Database

SQL Server in the cloud

- No administrative overheads
- Shared or Reserved (Dedicated) Hardware
- High Availability
  - database is up and running 99.99% of time
  - Azure automatically handles critical servicing tasks, such as patching, backups, Windows and SQL upgrades
- Pay-as-you-grow pricing
- Provide a familiar development model

7.3.2 CosmoDB

A fully managed, highly scalable, query-able, schema-free document database, delivered as a service, for modern applications

- Query against Schema-Free JSON
- Multi-Document transactions
- Tunable, High Performance
- Designed for cloud first
- Multi-model NoSQL
  - key-value, graph, column-family, and document
  - Automatic Indexing
  - Guarantees less than 10-ms latencies for both, reads and writes at the 99th percentile, all around the world

8 Serverless computing

IaaS (Infrastructure as a Service) provides virtual servers available in the Cloud, they are bare machines.

PaaS (Platform as a Service) provides virtual application platforms (e.g., Python Flask, Node JS Express and JAVA Spring Boot)

- removes the need to manage the Operating System;
- keeps the requirement to manage scalability and package a complete application for deployment to the platform;
- does not provide the level of automation required for running a large number of microservices;
- requires container orchestration platforms like Kubernetes, Docker Swarm and OpenShift as PaaS solutions.

Serverless Computing provides compute resource abstraction to its logical conclusion.

It allows developers to build and run applications without managing the underlying infrastructure. It is a cloud-computing service model (another) where the cloud provider automatically manages the servers.

Instead of provisioning, scaling, and maintaining servers, developers focus on writing code, while cloud providers take care of the infrastructure.

8.1 Backend as a Service (BaaS)

Serverless was first used to describe applications that significantly or fully incorporate third-party, cloud-hosted applications and services, to manage server-side logic and state.

1. cloud databases and message brokers (e.g.: Parse, Firebase, ...)
2. authentication services (e.g.: Auth0, AWS Cognito)

These types of services have been previously described as "**(Mobile) Backend as a Service" (BaaS)**, mobile because services to back up mobile applications are mainly provided.

- Traditional approach: Develop and deploy your own backend services.
- BaaS approach: Compose your application using ready-to-use backend services provided by a cloud provider.

Aspect	Traditional Approach	BaaS
Backend	Developed by the application developer	Functionality provided by the cloud provider
Infrastructure	Must be configured and managed, at least partially	Managed by the provider
Deployment	Requires backend deployment	Not required for BaaS services
Scaling	Must be configured or delegated to the platform	Managed by the provider
Database	Must be implemented or integrated	Available as a service
Authentication	Must be implemented or integrated	Available as a service
Control	High	Depends on the features provided
Portability	Generally higher	Potential vendor dependency

Pro

- Reduced Development Effort
- Reduced Infrastructure Management
- Scalability
- Faster Time to Market

Cons

- Limited Control: Developers have less control over the implementation and configuration of the backend.
- Vendor Lock-in: The application may become dependent on the provider's specific APIs and features.
- Security: The application relies on the provider's security practices and data protection policies.
- Cost: Costs may increase as usage grows and depend on the provider's pricing model.

8.2 Function as a Service (FaaS)

Serverless can also mean applications where the server-side logic is still written by the application developer, but, unlike traditional architectures, it runs in compute containers in form of functions.

These containers/functions (usually) are:

- **event-triggered** (they are executed in reaction to an event),
- **ephemeral** (may only last for one invocation), and
- **fully managed** by a third party (generally, the cloud provider).

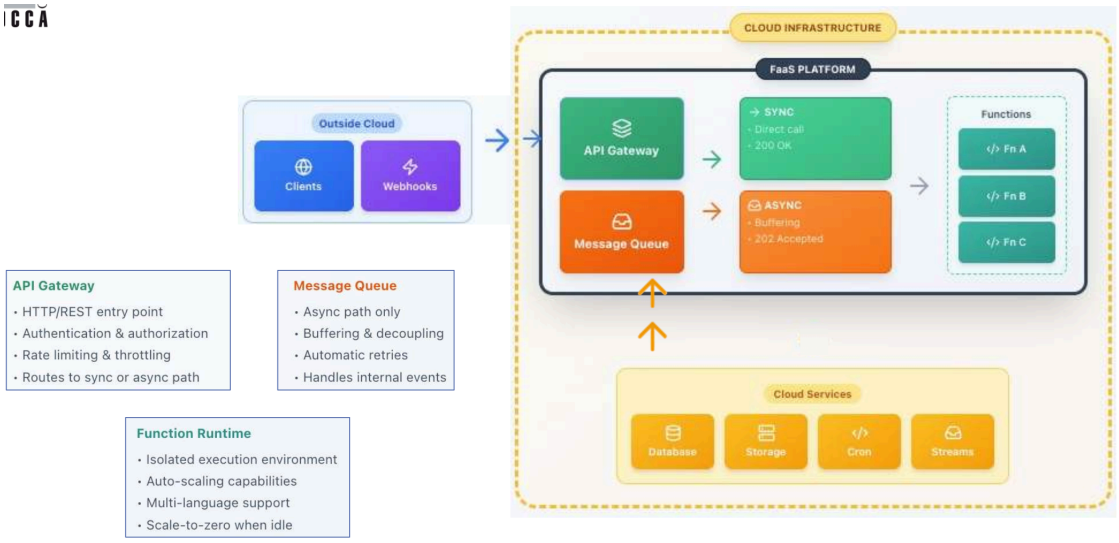


Figure 21: Function as a Service (FaaS) architecture

> By using BAAS and FAAS, we obtain serverless computing

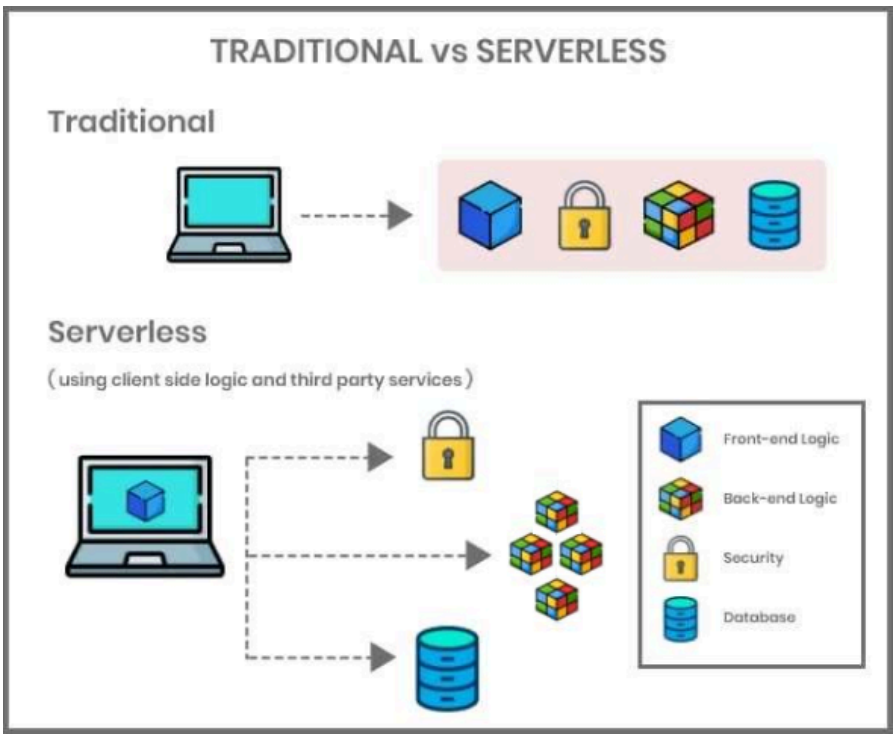


Figure 22: Serverless computing architecture

8.2.1 FaaS enabler: API Gateway

An API gateway is an HTTP server where routes are defined in configuration, and each route is associated with a resource to handle (handler) that route. In a Serverless architecture such handlers are often FaaS functions.

1. When an API gateway receives a request, it finds the routing configuration matching the request, and, in the case of a FaaS-backed route, will call the relevant FaaS function.
2. The FaaS function will execute its logic and return a result to the API gateway, which in turn will transform this result into an HTTP response that it passes back to the original caller.

Beyond purely routing requests, API gateways may also perform authentication, input validation, response code mapping, and more.

Amazon's API Gateway is a BaaS service, that you configure, but do not need to run or provision yourself.

8.2.2 State

> FaaS functions have significant restrictions when it comes to **local state**. Local state=data stored in variables in memory, or in a local disk

There is **no guarantee** that such state is persisted across multiple invocations.

FaaS functions are described as stateless, but it's more accurate to say that any state of a FaaS function that is required to be persistent needs to be externalized outside of the FaaS function instance.

8.2.3 Time (duration and startup)

FaaS functions are typically **limited** in how long each invocation is allowed to run. AWS Lambda function at most 15 minutes, before being terminated.

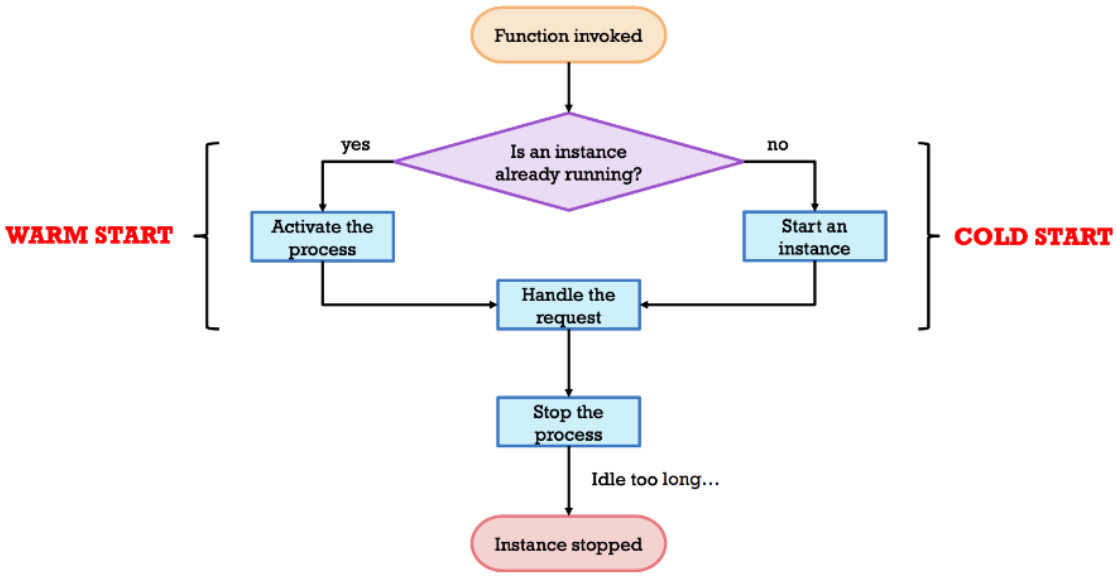
> Long-lived tasks are **not suited** to FaaS functions Trick: you may create a chain of several different FaaS functions, whereas in a traditional environment you may have one long-duration task

It takes some time for a FaaS platform to initialize an instance of a function before each event (from a few milliseconds to several seconds). Initialization of a Lambda function will either be a:

- warm start: reusing an instance of a Lambda function from a previous event.
- cold start: creating a new instance, starting the function, host process, etc.

The Cold start could be a problem depending on:

- the programming language used to create the function,
- how many libraries are used in the function,
- how much code you have in the function,
- the configuration of the function environment



8.2.4 pros and cons

Benefits of FaaS:

- Reduced operational cost (Serverless is, at its most simple core, an outsourcing solution)
- Reduced development cost
  - A BaaS database removes much of the database administration overhead
  - Reduced packaging and deployment complexity
  - Reduced time to market and high flexibility
- Reduced execution costs
  - unless you have a very steady traffic shape that uses the whole capacity of your server hosts, then you can save money using FaaS
- Greener computing
  - Reduce inefficiency

Drawbacks of FaaS:

- Vendor control – Vendor Lock-in
  - system downtime, unexpected limits, cost changes, loss of functionality, forced API upgrades
- Multitenancy problems
  - Security, robustness, performance
- Limitations in the function duration
- Startup latency
- Testing, Debugging, Monitoring
  - Currently difficult or limited

8.3 Amazon Lambda

1. **No servers to provision (nor roles, permissions, and other settings)**

   > «AWS Lambda lets you run code without provisioning or managing servers.»

   FaaS is about running backend code without managing your own server systems or your own long-lived server applications. A server isn't needed, nor an application (in the server) that is running constantly. No Server, VM, Container for long running applications
2. **Sub second metering**

   > «You pay only for the compute time you consume - there is no charge when your code is not running»

   With AWS Lambda, you are charged:
   - for every 100ms your code executes and
   - number of times your code is triggered.

   You don't pay anything when your code isn't running.
3. **Language agnostic**

   > «With Lambda, you can run code for virtually any type of application or backend service - all with zero administration».

   FaaS offerings do not require coding to a specific framework or library. FaaS functions are regular applications. AWS Lambda functions can be implemented in Javascript, Python, Go, any JVM language (Java, Clojure, Scala, etc.), or any .NET language. Or any language that can compile down to a Unix process or a Docker container.
4. **Resource provisioning**

   > «Just upload your code and Lambda takes care of everything required to run»

   Deployment is very different from traditional systems since we have no server applications to configure. In a FaaS environment, we upload the code for our function to the FaaS provider, and the provider does everything necessary for provisioning resources, instantiating VMs, managing processes, etc.
5. **Automatic scalability**

   > «and scale your code with high availability.»

   Horizontal scaling is completely automatic, elastic, and managed by the provider. If your system needs to be processing 100 requests in parallel the provider will handle that without any extra configuration on your part. With FaaS the vendor handles all underlying resource provisioning and allocation—no cluster or VM management is required by the user at all.
6. **Events**

   > «You can set up your code to automatically trigger from other AWS services»

   Functions in FaaS are typically triggered by event types defined by the provider. Amazon AWS includes
   - S3 (file/object) updates,
   - time (scheduled tasks),
   - and messages added to a message queue.
7. **http requests**

   > «or call it directly from any web or mobile app.»

Most providers also allow functions to be triggered as a response to an HTTP requests by using a component called API gateway.

8.4 Serverless vs PaaS

The difference between Serverless Computing and Platform as a Service (PaaS) lies in the level of abstraction, scaling, and billing.

8.4.1 Level of Abstraction

- PaaS:

  The cloud provider offers a platform where developers can deploy and run their applications, but they still need to configure and manage parts of the infrastructure. Developers control the runtime, but the underlying hardware and operating system are managed by the provider.

- Serverless:

  Serverless takes abstraction a step further. Developers only write functions or small services, and the cloud provider manages everything else—including scaling, infrastructure, runtime, and operating systems.

8.4.2 Scaling

- PaaS: Applications need to be scaled either manually or automatically, but developers are responsible for ensuring the application can handle scaling. There is typically a base resource allocation that runs continuously, even during low traffic periods.
- Serverless:

  Serverless systems scale automatically and dynamically with workload demands. They can scale to zero when there is no traffic, meaning you don't pay for idle resources. Serverless is event-driven—functions only run when invoked, and the cloud provider manages scaling without any intervention from the developer.

8.4.3 Billing

- PaaS:

  PaaS charges you based on the resources allocated, whether they are fully utilized or not. This means you may still incur costs when your application is running but not being actively used.

- Serverless:

  In serverless, you pay only for the actual compute time (i.e., when your code is executed). There are no charges when your function is idle, leading to potentially significant cost savings for applications with variable loads.

8.5 summary

Serverless is good for:

- short-running
- stateless
- event-driven

examples of applications:

- Microservices
- Mobile Backends
- IoT
- Modest Stream Processing
- Bots, ML Inferencing
- Service integration

Serverless is not good for:

- long-running
- stateful
- data crunching

examples of applications:

- Databases
- Deep Learning Training
- Big Data Analytics
- Heavy-Duty Stream Analytics
- Numerical Simulationf(x)
- Video Streaming

9 Towards a world of services

The current trend in the IT world is to move from a world of products to a world of services. This is a **paradigm shift** that is affecting the way we design, develop and deliver software applications.

9.1 Internet of Things (IoT)

The **Internet of Things** is a system of physical objects that can be **discovered**, **monitored**, **controlled**, or **interacted** with by electronic devices that communicate over various networking interfaces and **eventually can be connected to the wider internet**. IoT deals with **ubiquitous communication and connectivity**:

- Older definitions focus mostly on connectivity and sensory requirements for entities involved in typical IoT environments.
- New definitions focus give more value to the need for ubiquitous and autonomous networks of objects where identification and service integration have an important and inevitable role.

In IOT, **services have a central role**

> **Definition 9.1** *(Service (informal)).* A **service** is a program that performs specific tasks or provides value to users or other systems, typically over a network.

Iot can be seen as a **servitization** of the real world:

- The transformation of physical objects and real-world environments into service-based systems by embedding connectivity, sensors, and smart technologies.
- The notion of physical products evolves to provide continuous, data-driven services rather than being static, standalone entities.

We see things as a **service**:

- Things consume/expose services
- High number of components involved in single applications
- Dynamic (re)configuration of applications

9.2 Smart systems

In order to reach the goal of a world of services, we need to address several challenges and objectives in the design and implementation of smart systems:

- **Challenges**:
  - What is an Edge node? How does it connect to IoT and Cloud infrastructure?
  - Increasing capabilities/capacities of Edge resources
  - Data transfer to/from well-provisioned core data centers is expensive
  - Security/privacy concerns in moving data
  - Not all edge data is essential
  - How do we build a Smart System?
- **Objectives**:
  - Understand when and what data can be analyzed close to where it is captured (i.e., at the edges) – service/function orchestration
  - Effectively leverage resources close to the edge and in the network to support constraint-driven process (cost security, and QoS)

A system is any set of available variables selected by an observer to identity fundamental objects, the influential attributes of the objects, and the relationships of these objects that result in a phenomena.

Basic assumptions:
- Objects can be tangible or intangible
- Objects have attributes
- There are relationships amongst the objects
- There are relationships amongst the object attributes

9.2.1 Service computing

Services Computing is a field of computer science that focuses on designing, developing, deploying, and managing services to enable efficient and scalable interactions between software systems. A change was felt needed in systems design, deployment and maintenance compared to classical distributed architectures (web based monolithic apps).

Traditional (distributed) systems are single systems developed top-down by decomposition
- Generally, client-server
- Tight coupling: A single organization maintain/own the system

Contemporary distributed systems include third-party sub-systems
- E.g., a banking system does not offer only banking products to customers. It offers insurance contracts, event tickets, …
- Loose coupling: Each component is maintained independently from the others
- Each software component should be able to serve different systems in different ways

9.2.2 Software ecosystem

A Software Ecosystem consists of the **set of software solutions** that **enable, support and automate the activities** and transactions by the actors in the associated social or business ecosystem and the organizations that provide these solutions.

- Characteristics:
  - **Heterogeneous actors** (developers, users, organizations)
  - **Organic evolution** not centrally planned
  - **Fluid boundaries** continuously expanding
  - **Symbiotic relationships** between participants
  - **Distributed or market governance**
  - **Emergent innovation** from the bottom-up
- **Core Design Questions**
  - Interaction Capabilities: How to design components and enable them to interact?
  - Universal Interoperability: How to identify components that can interact in ANY environment?
  - Feature Advertisement: How can components advertise their features and characteristics?
  - Community Building: How to create a community of interacting individuals/components?
- **Service Components Context**
  - “Individuals” are service components implementing functionality
  - Features that need to be advertised include:
    - Interaction (interface and behavior)
    - Data exchanged between services
    - Non-functional aspects (technical specs, QoS, business rules, legal requirements)

But how can we support a software ecosystem?
- The use of protocols (HTTP) simplifies the access (consistent APIs)
- The use of media types and shared (JSON, XML) formats simplifies the data processing
- The use of components (Microservices) simplifies the identification of what functionalities a service provides and requests
- The use of containers simplifies the management issues (and provide required properties)

What is missing toward **dynamic systems**? (e.g., automatic discovery, composition, and execution)

- Let machines understand the **semantics of software** and associated properties
- Machine readable descriptions that can support [semi]automatic discovery, adaptation, composition and execution

9.2.3 Semantic Annotations

Semantic Web Services are too difficult to manage (mainly for the required skills and human effort)

- We need a simpler description format to annotate properties at semantic level to support machine interpretation and composition
- Associate contracts, defined as profiles, with services?

A profile is defined not to alter the semantics of the resource representation itself, but to allow clients to learn about additional semantics (constraints, conventions, extensions) that are associated with the resource representation, in addition to those defined by the media type and possibly other mechanisms.

Current initiatives of interests

- Open API Initiative (OAI) specification based on JSON and used by the Swagger-UI project to display the API and Swagger-Codegen to generate clients in various languages
- Dublin Core Application Profile (DCAP) guidelines defines metadata records which meet specific application needs while providing semantic interoperability with other applications on the basis of globally defined vocabularies and models.

9.3 Edge vs. Cloud computing

As data volume and velocity increases, we need to rethink our Cloud/Data Centre architecture. Move away from centralised solutions to more Peer-2-Peer Edge Clouds (Distributed Clouds). How do we support service provisioning and orchestration at the Network Edge?

Challenges:

1. Increasing capabilities/ capacities of Edge resources
2. Data transfer to/ from well-provisioned core data centers is expensive
3. Security/privacy concerns in moving data
4. Not all edge data is essential

objectives:

- Understand **when** and **what data** can be analyzed close to where it is captured (i.e., at the edges) → service/function orchestration
- **What is an Edge node**? How does it connect to IoT and Cloud infrastructure?
- Effectively **leverage resources close to the edge** and in the network to support constraint-driven process (cost security, and QoS)

9.4 API economy

The **API Economy** refers to the **monetization, exchange, and reuse of digital assets and capabilities** through Application Programming Interfaces (APIs). It is an **economic system** in which organizations expose their data, services, and processes via APIs, allowing external or internal developers, partners, and ecosystems to build new value-added products and services. Cloud Computing Freely available application programming interfaces (APIs), openly available data, were among the early drivers of this paradigm.

A new as-a-service economy is emerging with value being provided and consumed via APIs:

- Cloud, mobile and social are fueling the hyper-growth of API-centric, business-as-a-service economies

- Companies that were born on the web are co-creating value with others through APIs at an accelerated pace; enterprises are starting to explore them
- Agile, scalable, and consumable business as-a-service will shift the business services market similar to the shift of IT delivery through cloud

Key characteristics of the API economy:

- **APIs as Products**: APIs are treated as standalone products, often monetized directly or indirectly.
- **Value Creation:** APIs enable businesses to share services or data, leading to new opportunities.
- **Ecosystem Building:** APIs create interconnected systems of businesses, developers, and customers.
- **Composability**: Digital services are composed dynamically from multiple APIs, enabling rapid innovation and modular architectures.
- **Innovation Enabler:** External developers can build novel applications using existing APIs.
- **Revenue Generation:** Businesses monetize APIs via subscription, pay-per-use, or partnerships

Benefits:

For Businesses	For Developers	For Consumers
New revenue streams	Access to advanced tools	Seamless digital experiences
Faster innovation	Reduced development effort	Enhanced app functionality
Increased market reach	Opportunities for creativity	Integrated services in one place

Figure 24: Benefits of the API economy

Challenges:

- **Security Risks**: APIs expose data that can be exploited if not secured.
- **Complex Management**: Scaling and maintaining APIs in large ecosystems.
- **Monetization Balance**: Offering free vs. paid tiers for profitability.

One should distinguish between the use of APIs and implementing a service oriented architecture (SOA). SOA is a design paradigm that focuses on structuring software systems as a collection of loosely coupled services, while APIs are the interfaces through which these services are accessed.

SOA Service	API
Consumers are Internal (and maybe partner) developers	Consumers are Internal and External developers
Promote reuse within a company and sometimes with partners	Embracing of open community/social business is critical
SOAP & protocol independent headers	REST, leverage HTTP for Internet scale
Interoperability and tooling consumption based on WSDL	Easy of use based on simplicity and readability
Coarse grained	Fine grained, small amounts of data
Option for transactionality & reliability	Relaxed consistency
More extensive contract between service provider and consumer… in enterprise implementations	True 'black box' separation between Web API and consuming app; simple contract

Figure 25: SOA vs API

9.5 System of systems

A **System of Systems** is a collection of independent, self-contained systems that interact to create a larger, more complex system that achieves objectives or functionalities beyond the capabilities of the individual systems.

Key Characteristics:

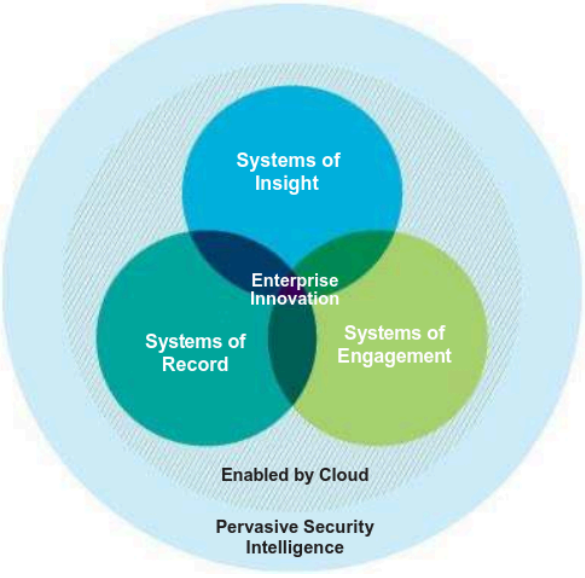
- Each constituent system is **operationally and managerially independent**.
- **Distributed control** — no single authority governs all subsystems.
- **Emergent behavior** arises from system interaction.
- Typical in **cyber-physical or critical domains** (defense, transport, smart cities).

Goal:

- To achieve emergent capabilities through the collaboration of autonomous systems.

How a global system can be organized?

" … Different application types will reside where the delivery model best supports client expectations, risk, compliance, service continuity and regulatory issues. **This model moves IT away from the traditional, on-premises, full-control model toward a distributed computing model** … "



[Enterprise] IT systems tend to be composed of complementary Layers:

- **Systems of engagement**: systems designed to facilitate interaction, collaboration, and communication between people, often focusing on creating personalized, user-centric experiences.
  - The external, visible part of a system
  - Focus on people and products, not processes
  - Lightweight, dynamic
- **Systems of records**: systems designed to manage and maintain structured data and support key business processes such as transactions, accounting, and compliance.
  - The internal, invisible part of a system
  - Focus on business processes, and data (transactions, accounting…)
  - Heavyweight, static
- **Systems of automation**: systems designed to automate tasks and processes, reducing or eliminating the need for human intervention.
  - Focus on connecting and automating the physical world

Each of the layers (Record, Engagement, Insight, Automation) is a system in its own right. They operate independently, often with different governance, technologies, and lifecycles. The enterprise as a whole functions as a System of Systems, where coordination yields emergent capabilities.

9.5.1 The Systems of Record

"Systems of Record" (SoR) are the traditional enterprise systems and applications that **handle business transactions** and other artifacts that constitute all the enterprise records.

- Designed to provide core **transactional systems**.
- **Databases** are at the center of these systems.
- **OLTP** and **Business Intelligence** are key drivers.
- The **network** is viewed primarily as a transport mechanism.
- **Essential** to day-to-day business operations.
- Must meet **regulatory compliance** requirements.
- High **QoS requirements** for performance and availability.

9.5.2 Systems of Engagement

The System of Engagement describes how IT should interact in a similar way to how consumers and employees do — with context-rich, immediate, and intuitive digital experiences.

"Empower customers, employees, and partners with context-rich apps and smart products to help them decide and act immediately in their moments of need."

- Improve **interactions with customers** to meet heightened expectations.
- Enable **self-service** to increase customer satisfaction.
- Attract more business by creating **new business models**.

Systems of Engagement vs Systems of Record

Consideration	Systems of Record	Systems of Engagement
Focus	Transactions	Interactions
Governance	Command & Control	Collaboration
Core Elements	Facts & Commitments	Ideas & Nuances
Value	Single Source of Truth	Discovery & Dialog
Standard	Accurate & Complete	Immediate & Accessible
Content	Authored	Communal
Primary Record Type	Documents	Conversations
Searchability	Easy	Hard
Usability	User is trained	User "knows"
Accessibility	Regulated & Contained	Ad Hoc & Open
Retention	Permanent	Transient
Policy Focus	Security (Protect Assets)	Privacy (Protect Users)

9.5.3 Systems of insight

A System of Insight analyzes data and generates actionable intelligence to guide decision-making processes. It collects, process, and interpret data from various sources to deliver insights that organizations can use to improve operations, customer experiences, or strategic outcomes. Systems of Insight bridge the gap between data and action.

Loop of activities:

1. discover the insights that matter most;
2. embed them into the software their customers and employees use to engage; and
3. continuously measure and learn from the results.

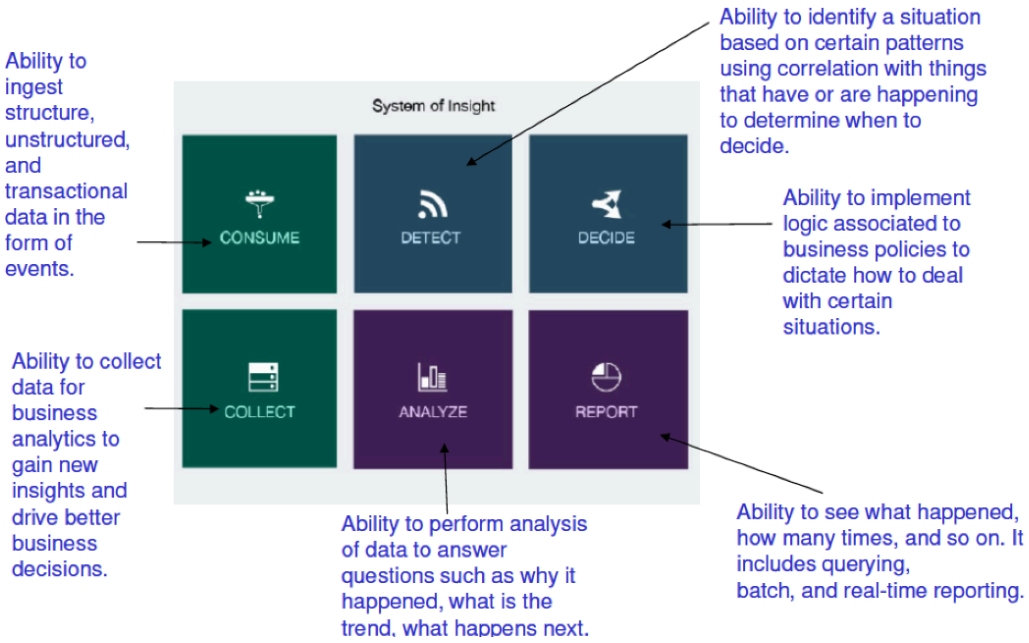


Figure 28: Building blocks of a system of insight

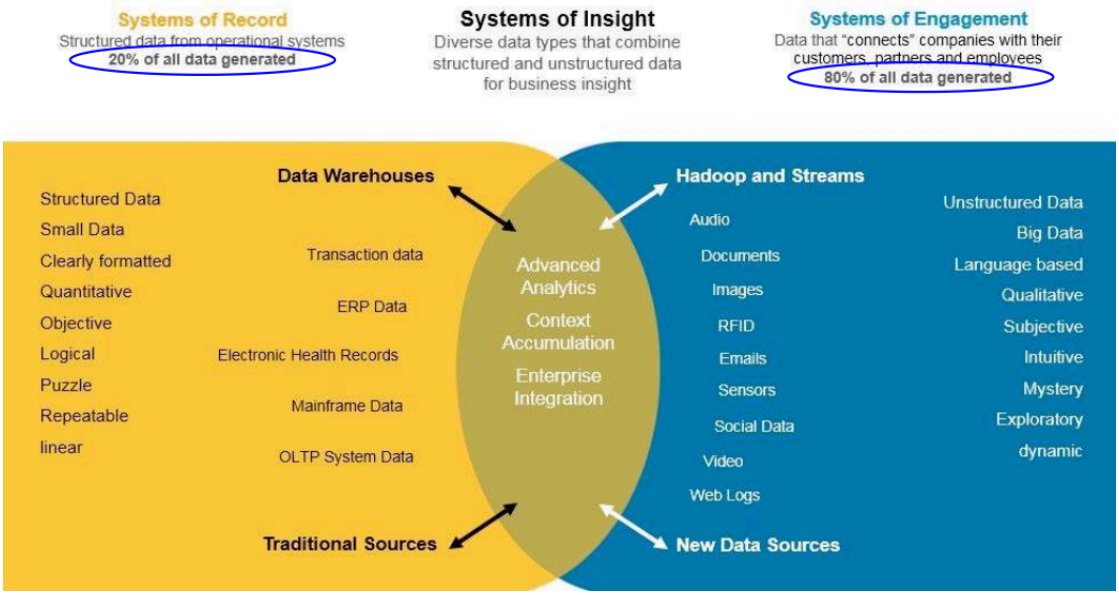


Figure 29: Amount of data generated by each system

Technical elements to build a system of insight

- **Small cross-functional insights teams**, sponsored by a leader with accountability for a business outcome. These teams will combine business domain expertise, data science skills, engineering expertise, and software development know-how to find, test, and implement insights.
- **A repeatable insight-to-execution process** that integrates data science, collaboration, governance, and agile implementation workflows. Without a repeatable process you won't have a system, just a bunch of people sifting through a sea of data for occasional nuggets.
- **A digital insights architecture** that will be built off the big data management and agile BI work you have done. It will add insights delivery engines and data feed management capabilities and provide a pluggable fabric for insight discovery, collaboration, and governance tools.

9.6 Service science

Services Science is the interdisciplinary application of science, engineering, and management for the purpose of improving services.

Services science also contributes to systematic innovation and improved productivity and is the guiding force for the improvement of services through improved predictability in the productivity, quality, performance, compliance, development, reusability of knowledge, and operational innovation in services.

A **service system** integrates people, organizations, technologies, and information resources to co-create value. The system's ability to scale depends on how these resources are combined to deliver competence as a service. Service systems integrate organizations, people, technology, and knowledge resources in different proportions to deliver value.

What makes Services Science hard is that it is **multidisciplinary**.

- Services depend critically on people, technology, organizations, and **co-creation** of value
- People **work together** and with technology and with organizations to provide value for clients
- **Shared information** helps coordinate activities – language, laws, measures, models, etc.
- So a **service system** is a complex **socio-techno-economic** system
- Growth requires innovation that combines **people, technology, organizations, value, shared information, clients**

- A service system is a value coproduction configuration of people, technology, internal and external service systems connected by value propositions, and shared information
- Services systems are both designed (Artificial) and shaped by evolutionary forces (Natural)

Services science aims to **research services** in the same **scientific manner** that spurred success in the manufacturing industry, and to make it easier to increase productivity through a scientific approach to modeling. At the same time, Services Science also **aims to make it easier for both clients and providers to forecast the future effects and risks of introducing services**, and to allow rational sharing of these effects and risks.

A Service Science professional collaborates with scientists, engineers, managers, and designers in interdisciplinary teams. To work effectively, they must combine **broad communication and collaboration skills** with deep expertise in a specific domain.

To Ensure that new service implementations or adaptations of existing environments consider the IT alternatives and factors such as cultural and human system dynamics. He/she must be able to analyze and model large and/or complex service systems across various sectors of the service economy. Demonstrates customer interaction skills equivalent to a business consultant seeking to improve productivity, quality, regulatory compliance, or innovation of a service system. Sustains effectiveness by ongoing discovery, analysis, and implementation of emerging service innovation frameworks and tools (lifelong learning).